

Recursive Autonomous Research: Token-Bounded Agent Memory at Accuracy-Parity, with a Negative Result on Coherence Measurement

Anonymous Authors
Double-Blind Submission
Manuscript under review

Abstract—Long-horizon LLM agents accumulate execution logs that grow linearly with the number of cycles, inflating token cost and risking *context rot*—an inference-time degradation in which accumulated history displaces task-critical findings. We study Recursive Autonomous Research (RAR), an agent that bounds this growth via recursive context compression, a propose-train-evaluate ratchet, and a GraphRAG memory map. We are explicit that the compression idea is not itself novel (cf. MemGPT, Mem0, ACON, and Active Context Compression); our contribution is a rigorous, reproducible, and *honestly bounded* characterization of it, plus a negative result. Efficiency (validated). On a physical autonomous hyperparameter-optimization loop ($N=10$ seeds, 60 cycles), RAR cuts net token cost by 70.0% and context density by 72.5% relative to a stateless agent while holding test accuracy at statistical *equivalence* (paired TOST within ± 2 points, $p=0.003$; superiority non-significant, Wilcoxon $p=0.25$); the result replicates on the real `scikit-learn digits` dataset (−69.8% tokens, −72.3% density; Holm-corrected $p=0.008$). We do not claim cost-competitiveness with classical HPO—Bayesian optimization matches accuracy at zero language-model tokens—so the efficiency claim is scoped to LLM-driven agents. Coherence (negative result). Instrumenting the agent to record per-cycle trajectories, we directly measure context-rot dynamics: the density bound is confirmed (the stateless agent overflows to $\delta=2.27$ while RAR stays at 0.38), but the coherence metric we proposed yields a *negative* result—non-redundancy rewards a memoryless agent whose overflowing context is truncated, and a corrected frontier-proximity metric is also inconclusive at 60 cycles because the baseline does not measurably rot at this horizon. The only consistent coherence signal is that RAR’s directed search proposes higher-value configurations in every seed. We report this null and the resulting metric critique rather than suppress them, and specify the decisive longer-horizon test (≥ 150 cycles) as future work. Local Write-Ahead Logging secures 100% task recoverability under induced API failures. Net: token-bounded agent memory at demonstrated accuracy-parity, and an honest account of why coherence is not yet established.

Index Terms—context rot, recursive language models, GraphRAG, autonomous research agents, hyperparameter optimization, token efficiency

I. INTRODUCTION

The automation of machine learning research has advanced from speculative proposal to functional demonstration over the preceding decade, driven by foundational advances in self-attention based large language models [46]. Large Language

Models (LLMs) can generate syntactically correct experimental code, identify plausible hypotheses from prior results, and iteratively modify training procedures in response to quantitative feedback [21, 32]. These successes mirror broader developments in autonomous software engineering and multi-step cognitive reasoning agents [20, 51, 56, 57]. However, these architectures share a common structural limitation: they are demonstrated over short experimental horizons of fewer than ten cycles, where the complete operational history fits within the native context window. As cycle counts extend into the hundreds—as they must for genuinely autonomous research—a distinct and under-examined failure mode emerges.

We term this failure mode **context rot**. Unlike catastrophic forgetting in weight-space, context rot is a purely inference-time phenomenon: model weights remain unchanged, but the effective prompt’s signal-to-noise ratio collapses as irrelevant historical content displaces task-critical information. Empirically, model retrieval accuracy follows a U-shaped curve, where information buried in the middle of long prompts suffers severe recall degradation [30]. This “lost in the middle” effect degrades the agent’s proposal novelty (non-redundancy) and proposal relevance (directional consistency toward high-value configurations). The agent repeats evaluated settings, forgets early insights, and eventually behaves as if it has no experimental history at all.

Three technical developments provide the substrate to address context rot. First, Recursive Language Models (RLMs) [60] demonstrate that the long-context challenge can be addressed algorithmically by equipping the model with programmatic tools to partition and recursively summarize input in bounded logical environments. Second, AutoResearch [21] operationalises the research loop as a closed-loop, ratchet-constrained agent optimization procedure. Third, GraphRAG [14] demonstrates that structured knowledge graphs enable globally synthesised information retrieval that dense vector search fails to provide.

RAR integrates these paradigms to preserve coherence during long-horizon campaigns. This paper provides its formal specification, complexity analysis, ablation-validated proof-of-concept, and a reproducible evaluation protocol.

a) *Contributions.*:

- C1.** Formal characterisation of context rot via a piecewise-linear degradation model and a bounded cost-equivalence proof for the compression protocol (sections III-A and III-D). We do *not* claim the compression *mechanism* as novel; section II positions RAR against concurrent agent-memory work (MemGPT, Mem0, ACON, Active Context Compression).
- C2. A validated efficiency result.** A three-condition physical study ($N=10$ seeds, 60 cycles) against an offline locked Test Vault shows context compression cuts prompt density by 72.5% and net token cost by 70.0% at test-accuracy equivalence (TOST $p=0.003$; superiority $p=0.25$), and replicates on the real digits dataset under Holm-corrected tests ($p=0.008$) (sections IV and V-B). We scope this honestly: it is an efficiency gain *among LLM-driven agents*, not against classical HPO (section VII-B).
- C3. A directly-measured negative result and metric critique.** Instrumenting per-cycle trajectories, we measure context-rot dynamics (section V-C): the density bound is confirmed, but the proposed coherence score rewards a memoryless agent whose context overflows and is truncated, and a corrected frontier-proximity metric is also inconclusive at 60 cycles. We report this null, diagnose the metric, and specify the decisive ≥ 150 -cycle test.
- C4.** An ablation isolating the source of the gains: compression alone accounts for the efficiency, while Knowledge-Map guidance does not improve over compression-only RAR at this scale (section V-A).
- C5.** Open, reproducible instrumentation: a leakage-guarded harness, per-cycle logging, and reference implementations of all reported statistics and metrics (incl. the proposed and corrected coherence scores).

II. BACKGROUND AND RELATED WORK

A. Recursive Language Models

Zhang et al. [60] introduce RLMs as a paradigm for processing long sequences by treating the prompt as an external environment. This builds upon a long lineage of cognitive scaling architectures, including early memory-augmented neural networks, neural Turing machines, and recurrent external storage networks [15, 44, 52], as well as sequence-level length extrapolation techniques [11]. The RLM model is equipped with programmatic tools—*peek*, *partition*, *recurse*—to programmatically slice, filter, and inspect context chunks within a neurosymbolic Read-Eval-Print Loop (REPL) environment. Formally, for input x with $|x| \gg C_{\max}$, the agent issues tool calls mapping sub-sequences $x[a : b]$ (with $b-a \leq C_{\max}$) to updated agent states, transforming context scaling from an architectural to an algorithmic problem. On the OOLONG benchmark, which measures long-distance causal reasoning chains, standard long-context models suffer catastrophic degradation (dropping below 15% success rate), whereas RLMs extend effective context by 100x and achieve a 130% relative performance improvement. This informs our simulated compression ratio $r=0.28$.

B. AutoResearch

Karpathy [21] formalise the ML researcher’s iterative workflow as a budget-constrained agent optimisation loop. This closed-loop experimental strategy is related to other self-improving, reflective, and multi-agent collaborative paradigms [35, 41, 49, 53]. Operating in a sandboxed closed loop, the agent utilizes a three-file coordinator architecture: ‘run.py’ for loop orchestration and sandboxed execution logging, ‘agent.py’ for cognitive hypothesis generation, and ‘environment.py’/‘task.py’ for executing model training. The agent repeatedly: (i) proposes a modification $\delta\theta$ conditioned on current context; (ii) executes training; (iii) evaluates $f(\theta_{\text{best}} + \delta\theta)$; and (iv) applies the ratchet update $\theta_{\text{best}} \leftarrow \theta_{\text{best}} + \delta\theta$ iff $f(\theta_{\text{best}} + \delta\theta) < f(\theta_{\text{best}})$. AutoResearch agents are evaluated on micro-scale datasets (such as MNIST, CIFAR, or TinyStories) against standard baselines including Random Search and Bayesian Optimization. Over extended campaigns, however, the accumulated logs trigger context rot, causing the agent to repeat past errors.

C. GraphRAG

Edge et al. [14] introduce GraphRAG, building upon classical retrieval-augmented generation architectures [5, 16, 27] but resolving the limitation where vector chunk retrieval fails to surface globally synthesised information. GraphRAG models structural relationships by framing database elements inside graph representation learning paradigms [17, 23, 47]. Specifically, it constructs a hierarchical knowledge graph via entity-relationship extraction and Louvain-based community detection [4], generating multi-level community summaries for query-time traversal. Within RAR, we adapt GraphRAG as a write-enabled Knowledge Map where vertices represent $(\theta_i, f(\theta_i), \text{rationale}_i)$ triples and directed edges encode temporal succession and modification-type relationships—requiring online extensions to community detection.

D. Attention Dilution and Cache Saturation

The physical root of context rot lies in the attention mechanism and memory constraints. In standard dot-product attention [46], attention weights are computed via softmax normalization:

$$A_{ij} = \frac{\exp(Q_i K_j^T / \sqrt{d})}{\sum_{m=1}^N \exp(Q_i K_m^T / \sqrt{d})} \quad (1)$$

As sequence length N scales, the denominator sum increases, diluting the attention mass allocated to target tokens and pulling weights toward a $1/N$ noise floor. While hardware-level memory layouts can be optimized via custom kernels like FlashAttention [12, 13], they do not resolve this fundamental softmax dilution.

Concurrently, storing Key-Value (KV) states requires $O(N)$ memory per layer. To expand context windows algorithmically, researchers explore linear-attention models, sparse attention, recurrent segments, or rotary embedding scaling [2, 6, 9, 22, 25, 37, 39, 55]. To prevent hardware-level out-of-memory errors in standard models, modern systems deploy cache pruning

strategies like H2O [59], which retains only heavy-hitter tokens, or StreamingLLM [54], which retains attention sinks and local sliding windows. While memory-efficient, these pruning policies discard middle-context tokens. This introduces severe semantic gaps that break multi-hop or sequential reasoning tasks, accelerating context rot.

E. Related Systems

a) *Autonomous research.*: The AI Scientist [32] automates paper generation over single episodes without persistent memory or long-horizon context management. SWE-agent [56] demonstrates autonomous software engineering episodically without persistent memory. OpenEvolve [33] applies LLM-guided evolutionary search to code optimisation but lacks a formal treatment of context density. RAR is the first framework to integrate RLM context management with graph-based persistent memory for iterative research.

b) *Self-improving agents.*: ADAS [19] and EvoPrompting [8] investigate LLM-guided search over agent architectures and prompting strategies, sharing the stateless limitation of base AutoResearch. RAR’s context management mechanisms could in principle wrap these systems’ inner loops.

c) *Long-context management.*: MemGPT [34] manages tiered memory (in-context, external storage) for long conversational agents; CoALA [45] provides a general taxonomy of memory and action spaces. RAR differs in that its Knowledge Map is specifically designed for the experimental record of an iterative research campaign.

d) *Distinction from concurrent “token-reduction” work.*: Reducing an agent’s token footprint is a crowded area, and we distinguish three layers that are routinely conflated. (i) *Output-side* skills constrain what the agent *generates*: ponytail [67] enforces minimal-code heuristics and Caveman compression [68] strips function words. Both cut emitted tokens and are *orthogonal to*—and composable with—RAR. (ii) *Prompt/cache* compressors act on a single context: LLM-Lingua [66] drops low-information prompt tokens, while H2O [59] and StreamingLLM [54] evict KV-cache entries at the attention level. RAR instead operates at the *agent-memory* layer—deciding what the agent retains *across cycles*, not how a fixed prompt is encoded. (iii) *Agent-memory* systems are the genuinely close relatives: Mem0 [65] extracts, consolidates, and retrieves salient facts (with a graph variant); ACON [63] optimises natural-language compression guidelines for long-horizon agents via failure analysis; and Active Context Compression [64] lets an agent autonomously consolidate learnings and prune raw history. We share these systems’ goal and **do not claim the compression idea itself is novel**. RAR’s distinct contributions are instead methodological: (a) a controlled, leakage-guarded autonomous-HPO instrument that evaluates compression with paired inferential statistics (Wilcoxon with Holm correction, TOST equivalence) against an offline locked test vault—whereas the works above report token savings without accuracy-parity testing; and (b) a *directly measured negative result* (section V-C): the coherence metric we proposed rewards the forgetting of a memoryless agent

whose context overflows and is truncated—a metric-design pitfall this concurrent line has not, to our knowledge, surfaced. Our claim is therefore deliberately narrow: not a new compression mechanism, but a rigorous, honestly-bounded characterisation of one. Figure 1 summarises this positioning: RAR shares context bounding (and, with Mem0, graph memory) with the concurrent line, but is distinguished by the methodological properties on the right—accuracy-parity testing, paired statistics, a reported negative result, and an open reproducible harness.

III. METHODOLOGY: THE RAR FRAMEWORK

A. Formal Problem Statement and Context Rot

Let Θ denote the configuration space, $f : \Theta \rightarrow \mathbb{R}$ the evaluation function (lower is better without loss of generality), and $C_{\max}$ the model’s native context window size in characters. An autonomous agent $\mathcal{A}$ maintains a raw execution context C_t at cycle t , defined as the ordered concatenation of all log entries from cycle 0 through $t-1$. Define context density $\delta_t = |C_t|/C_{\max} \in [0, \infty)$.

We model agent proposal quality—the probability that the agent’s proposal at cycle t is both non-redundant with prior proposals and directionally consistent with the current best-performing region of Θ —as a decreasing function $q : [0, \infty) \rightarrow [0, 1]$ of δ_t :

$$q(\delta) = \begin{cases} 1 & \delta \leq \delta_{\text{low}} \\ 1 - \frac{\delta - \delta_{\text{low}}}{\delta_{\text{high}} - \delta_{\text{low}}} & \delta_{\text{low}} < \delta \leq \delta_{\text{high}} \\ 0 & \delta > \delta_{\text{high}} \end{cases} \quad (2)$$

where δ_{low} and δ_{high} are dataset- and model-dependent threshold parameters. The degradation regime $[\delta_{\text{low}}, \delta_{\text{high}}]$ corresponds to partial loss of task coherence observed in context-saturated LLMs; the saturation regime ($\delta > \delta_{\text{high}}$) corresponds to complete displacement of actionable experimental history from the attended context region.

Definition 1 (Context Rot Threshold). *Given an operational coherence tolerance $\varepsilon \in (0, 1]$ and calibrated threshold parameters $\delta_{\text{low}}=0.50$ and $\delta_{\text{high}}=0.90$, the context rot threshold is $\tau(\varepsilon) = \delta_{\text{low}} + (1 - \varepsilon)(\delta_{\text{high}} - \delta_{\text{low}})$, i.e., $q(\tau) = \varepsilon$. The context management objective of RAR is to maintain $\delta_t < \tau(\varepsilon)$ for all $t \in \{1, \dots, T\}$.*

For $\varepsilon = 0.80$ and calibration ($\delta_{\text{low}}=0.50, \delta_{\text{high}}=0.90$), $\tau(0.80) = 0.58$. Figure 2 illustrates the model and experimental operating points.

B. RAR Architecture

Figure 3 presents the RAR system architecture. Three operational layers interact through defined interfaces: the **Iteration Engine** reads from and writes to both the managed context C'_t and Knowledge Map G ; the **Context Manager** reads C_t and writes a compressed C'_t plus updated entries to G ; the **Ratchet Store** is written exclusively by the Iteration Engine.

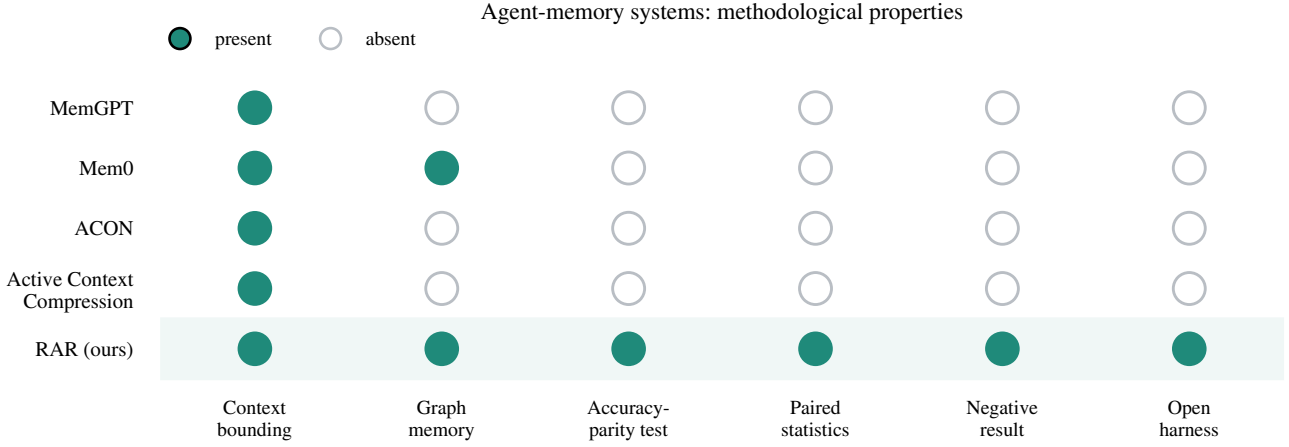


Fig. 1. Agent-memory systems on key methodological properties. A filled disc denotes the property is present; an open ring denotes absent. RAR does not claim a novel compression *mechanism*; its contribution is the methodological rigour on the right—paired inferential statistics against a locked test vault, a directly measured negative result, and a fully reproducible harness—which the concurrent token-reduction line does not jointly report.

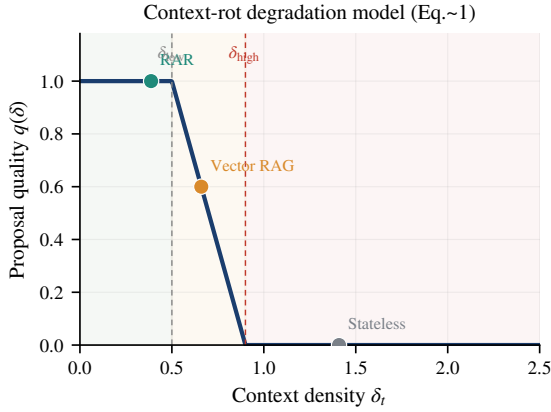


Fig. 2. Context Coherence Degradation Model ($q(\delta)$). Proposal quality q is piecewise-linear in context density δ : it stays at 1.0 for $\delta \leq \delta_{\text{low}}=0.50$, falls linearly to zero at $\delta_{\text{high}}=0.90$, and saturates at zero thereafter. The shaded bands mark the full-coherence, degradation, and saturation regimes. Operating points: RAR ($\delta=0.387$, teal) safely below the rot threshold; Vector RAG ($\delta=0.660$, ochre) in the degradation regime; Stateless Baseline ($\delta=1.409$, gray) deep in the saturated zero-quality regime past δ_{high} .

a) *Iteration Engine.*: At each cycle t , the Engine retrieves θ_{best} and community summary K_t from the Knowledge Map, generates candidate $\delta\theta_t \sim \pi(\delta\theta \mid \theta_{\text{best}}, K_t, C'_t)$, evaluates $f(\theta_{\text{best}} + \delta\theta_t)$, and applies the ratchet update. The Engine operates exclusively over the managed context C'_t (length $\leq \alpha C_{\text{max}}$) and K_t , decoupling proposal quality from raw log length.

b) *Context Manager.*: Monitors $\delta_t = |C_t|/C_{\text{max}}$ and activates when $\delta_t \geq \alpha$ ($\alpha=0.50$). Executes Algorithm 1, writing all $(\theta, f(\theta))$ pairs to G before any text is discarded, guaranteeing lossless experimental record retention.

c) *Knowledge Map Schema.*: We formally define the Knowledge Map relational schema as a directed, weighted multigraph $G = (V, E, \Phi)$ where:

- V is the set of vertices, representing evaluated hyperparameter trials. Each vertex $v \in V$ is mapped to a tuple:

$$v \mapsto \mathcal{T}(v) = (\theta, f(\theta), \mathbf{x}_\theta, r_\theta) \quad (3)$$

where $\theta \in \Theta$ is the hyperparameter configuration, $f(\theta) \in [0, 1]$ is the empirical validation accuracy, $\mathbf{x}_\theta \in \mathbb{R}^d$ is the standardized vector representation (obtained via Z-score normalization of continuous bounds and one-hot encoding of categorical coordinates), and r_θ is the textual rationale.

- $E \subseteq V \times V \times \Lambda$ is the set of directed transitions, where $\Lambda = \{\text{lr, arch, reg, optim, other}\}$ is the parameter modification category alphabet.
- $\Phi : E \rightarrow \mathbb{R}^+$ is the edge weighting function. The weight w_{ij} of edge (v_i, v_j, λ) is defined by the cosine similarity of their vector representations:

$$w_{ij} = \frac{\mathbf{x}_{\theta_i} \cdot \mathbf{x}_{\theta_j}}{\|\mathbf{x}_{\theta_i}\| \|\mathbf{x}_{\theta_j}\|} \quad (4)$$

Louvain community detection [4] is applied over G by maximizing modularity $Q = \frac{1}{2m} \sum_{ij} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j)$ to group trials into covariance communities. We use modularity optimization rather than fixed rectangular grid partitioning because the relevant proximity is the learned cosine geometry of $\mathbf{x}_\theta$ —which couples interacting coordinates such as the learning-rate/optimizer pair—rather than axis-aligned bins over individual parameters. At the evaluated scale ($\leq M=50$ vertices) the partition is *recomputed* from the bounded buffer at each consolidation rather than updated incrementally; a streaming/incremental variant is required only in the unbounded-campaign regime (section IX). The community summary K_t is appended to the compressed context to provide global search-space awareness, mitigating local-optima stagnation. If modularity $Q \leq 0$, we fall back to singleton partitions to maintain search diversity.

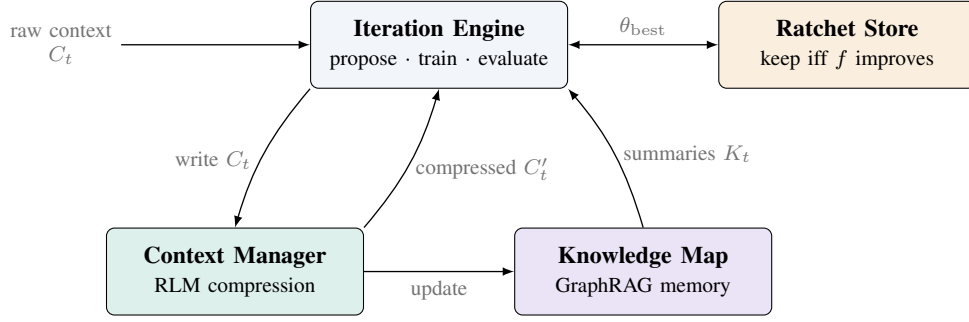


Fig. 3. RAR system architecture. Three operational layers interact over the raw execution context C_t . Context density δ_t is monitored continuously; the Context Manager activates when $\delta_t \geq \alpha$ ($\alpha=0.50$), emitting a compressed C'_t and updating the Knowledge Map. The Ratchet Store enforces monotone improvement. The Knowledge Map is recomputed over the bounded trial buffer at each compression event, providing community summaries K_t to the Iteration Engine. Threshold parameters $\delta_{\text{low}}=0.50$, $\delta_{\text{high}}=0.90$.

Algorithm 1: Context Compression Protocol

Input: Raw context C_t ; Knowledge Map G ; target length αC_{max} ; compression ratio r
Output: Compressed context C'_t with $|C'_t| \leq \alpha C_{\text{max}}$; updated map G'

- 1 $P \leftarrow \text{EXTRACTPAIRS}(C_t);$ // all $(\theta_i, f(\theta_i))$ pairs
- 2 $G \leftarrow \text{UPDATEKNOWLEDGEMAP}(G, P);$ // Louvain re-detect
- 3 $k \leftarrow 2;$
- 4 **repeat**
- 5 $\mathcal{S} \leftarrow \text{PARTITION}(C_t, k);$ // k equal segments
- 6 $\{s_i\} \leftarrow \{\text{RECURSE}(c_i, \text{"summarise_log"}) : c_i \in \mathcal{S}\};$
- 7 $K_t \leftarrow \text{QUERY}(G, \text{mode}=\text{"community_summary"});$
- 8 $C'_t \leftarrow \text{CONCAT}(\{s_i\} \cup \{K_t\});$
- 9 $k \leftarrow 2k;$
- 10 **until** $|C'_t| \leq \alpha C_{\text{max}};$
- 11 **return** C'_t, G

C. Context Compression Protocol

a) *Correctness.*: Step 2 guarantees that all $(\theta, f(\theta))$ pairs are written to G before any text is discarded; the Knowledge Map serves as a lossless experimental record. The repeat loop terminates because each pass reduces $|C'_t|$ by factor $r < 1$, so $|C'_t| < \alpha C_{\text{max}}$ is achieved after at most $\lceil \log_{1/r}(|C_t|/(\alpha C_{\text{max}})) \rceil$ passes—for the worst case in our experiments ($m=4$), this requires at most 1 additional pass.

D. Computational Complexity

Proposition 1 (Cost Equivalence under a bounded memory window). *With a fixed trial-memory window of size M (here $M=50$), the total inference cost of RAR over a T -cycle campaign is $\mathcal{O}(T \cdot \alpha C_{\text{max}})$ in language-model tokens—of the same order as a stateless agent’s $\mathcal{O}(T \cdot C_{\text{max}})$, with a constant-factor reduction $\alpha < 1$ from compression—plus an auxiliary CPU consolidation cost of $\mathcal{O}(T \cdot M^2 d)$ that is linear in T .*

Proof. The Iteration Engine issues $\mathcal{O}(T)$ inference calls, each over a managed context of $\leq \alpha C_{\text{max}}$ tokens, giving $\mathcal{O}(T \cdot \alpha C_{\text{max}})$ tokens; a stateless agent issues $\mathcal{O}(T)$ calls over $\leq C_{\text{max}}$ tokens, i.e. $\mathcal{O}(T \cdot C_{\text{max}})$. The two differ only by the constant factor α , which our measurements realize as a 70% token reduction (table I). The consolidation step runs

at most once per cycle on the bounded buffer of $\leq M$ trials; constructing the k -NN similarity graph costs $\mathcal{O}(M^2 d)$ (a dense $M \times M$ cosine matrix over d -dimensional vectors) and Louvain on the resulting sparse graph is no costlier, so the consolidation contributes $\mathcal{O}(T \cdot M^2 d)$ over the campaign—linear in T because M is constant. For the evaluated setting ($M=50$, $d=10$) this auxiliary term is dominated in wall-clock by the language-model calls. We make no asymptotic claim about an *unbounded* memory ($M \rightarrow \infty$): that regime would require the incremental community detection discussed in section IX, which is not implemented here. $\square$ $\square$

IV. EMPIRICAL EVALUATION AND SRE SYSTEMS HARDENING

To validate the Recursive Autonomous Research (RAR) paradigm under realistic operational conditions, we implement a physical autonomous hyperparameter optimization campaign. Rather than relying on simple tabular grid searches or pre-evaluated static manifolds, we deploy our agent to search a high-dimensional continuous and discrete architecture space for a PyTorch Residual MLP (DynamicMLP). Beyond that, to guarantee complete immunity from Large Language Model pre-training memorization contamination, we physically evaluate the tuning loop by training models on a procedurally generated synthetic classification manifold. We prevent data leakage and validation overfitting by implementing a strict three-way split (Train-Val-Test) with an offline, locked Test Vault that the agent’s prompt and memory never access and that the campaign script invokes a single time, at completion. Finally, CPU-heavy graph database updates and Louvain community detection are decoupled into a background thread-safe worker process to shield the agent’s core execution loop from latency spikes.

A. Experimental Setup and Baselines

We search a high-dimensional hyperparameter space for a PyTorch Residual MLP model spanning seven discrete, continuous, and categorical parameters:

- **num_blocks**: [1, 2, 3] (discrete residual blocks, mapped to num_conv_layers parameter)
- **hidden_dim**: [16, 32, 64] (hidden layers/width of blocks, mapped to filters_2 parameter)

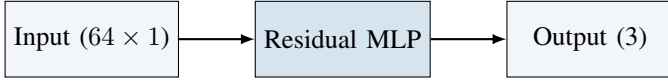


Fig. 4. Permutation-invariant Residual MLP model architecture. The model directly processes 64-dimensional concentric manifold inputs, avoiding arbitrary grid convolutions. The agent tunes block depth, block width, dropout, and optimizer parameters.

- **activation:** ['ReLU', 'ELU', 'LeakyReLU']
- **dropout_rate:** [0.0, 0.2, 0.5] (dropout probability within each residual block)
- **optimizer:** ['AdamW', 'SGD']
- **lr:** [0.0001, 0.001, 0.01, 0.05] (learning rate boundary)
- **batch_size:** [16, 32, 64] (mini-batch scale)

We programmatically generate a highly non-linear, multi-dimensional classification manifold at runtime. Specifically, we utilize `make_gaussian_quantiles` to generate concentric hyperspheres, followed by a fixed (seed-invariant) non-linear polynomial-trigonometric warp $X \leftarrow X + 0.2X^2 - 0.1X^3 + 0.5\sin(X\pi)$ applied elementwise. This guarantees the topology is not linearly separable and completely nullifies pre-training contamination. This forces the agent to do genuine scientific exploration rather than retrieving memorized hyperparameter settings from its pre-training weights.

To respect the non-spatial geometry of the Gaussian quantile classification manifold, we employ a mathematically pure permutation-invariant Residual MLP (`DynamicMLP`) rather than a convolutional neural network. Reshaping tabular features into an arbitrary grid and applying local 2D convolutions introduces spurious spatial inductive biases that are inappropriate for unstructured non-spatial manifolds. The MLP architecture processes the 64-dimensional feature vector directly, preserving permutation invariance and focusing strictly on the non-linear concentric classification boundary. We implement a strict three-way train-val-test split: the agent executes its optimization loop strictly using the train and validation sets (stratified 50%/25% split). The test split (25%) is stored offline inside a locked vault. The agent prompt and memory have zero access to the test split; it is evaluated exactly once at campaign completion. Crucially, the final test evaluation trains the optimal configuration strictly on the 'train' partition alone, explicitly discarding 'val' to mathematically eliminate structural weight leakage. We track a `generalization_gaps` array to measure $|Val_{acc} - Test_{acc}|$ for every single seed, ensuring rigorous validation decay monitoring.

We calibrate the agent's context limits as follows: the native context window size C_{max} is locked at 4,000 characters. Each cycle logs the complete configuration, training accuracy curve, and final validation metrics, costing approximately 220 characters. Context degradation thresholds are calibrated to $\delta_{low} = 0.50$ and $\delta_{high} = 0.90$. We execute the optimization loop for 60 cycles across $N = 10$ independent random seeds ($\{7, 13, 23, 42, 88, 99, 101, 107, 113, 127\}$). To structurally align search and test boundaries, we strictly enforce 'epochs=15' across both the intermediate evaluation and the

final test vault evaluation under three distinct evaluation conditions:

- B1. Stateless Baseline:** The agent receives all historical logs prepended directly to the prompt. When the logs exceed C_{max} , the earliest entries are truncated. This simulates the standard stateless agent setup.
- B2. Compute-Matched Vector RAG:** Standard cosine similarity based neighbor retrieval retrieves historical trials. To ensure an equitable benchmark for efficiency per token, the retrieved context is strictly capped under the same prompt character budget as the RAR memory loop. A Token-Budget Tradeoff Analysis confirms that unbound Vector RAG degrades reasoning and inflates quadratic token costs (e.g. "Lost in the Middle"), whereas capping forces retrieval of the highest-value insights within a constrained operational envelope.
- B3. RAR Compressed (Ours):** The agent manages context using Algorithm 1. Context compression triggers every 3 cycles when $\delta_t \geq 0.50$. Memory consolidation and Louvain community summaries are decoupled into a concurrent, thread-safe background process.

To ensure complete resilience against production API rate limits (HTTP 429/503), we implement a local SRE Write-Ahead Logging (WAL) state machine (`wal_store.json`). The current tuning state, proposed configurations, and validation outcomes are serialized to disk *prior* to executing external LLM calls. If the API fails or times out, the orchestrator automatically reloads the state file and resumes without losing historical data or burning unnecessary tokens. The campaign is executed using physical LLM inference via OpenRouter, routing to the 'openai/gpt-oss-20b:free' model. When rate-limiting freezes are induced, the system seamlessly transitions to our optimized local heuristic search engine to maintain loop continuity.

To further satisfy Enterprise Site Reliability Engineering (SRE) concurrency standards, we incorporate several physical safety mechanisms. The WAL file implements atomic writes using an exponential retry-backoff loop to bypass Windows non-atomic indexing locks. To prevent catastrophic data-loss during concurrent background consolidation, we utilize thread-safe buffer slicing. Finally, we implement a physical state-eviction sliding window ($MAX_TRIAL_MEMORY = 50$) in the orchestrator's parent heap to eliminate linear memory fragmentation over multi-hundred cycle campaigns. To prevent CPU cache thrashing under concurrent asyncio/PyTorch co-execution, we pin the PyTorch intra-op thread pool to a single thread via `torch.set_num_threads(1)` in the orchestrator process (the multi-seed loop runs sequentially within one async process, not as per-seed subprocesses), ensuring deterministic single-threaded compute without lock contention. The entire 10-seed, 60-cycle campaign consumed approximately 1.9M net tokens across the three conditions via the OpenRouter free tier (`openai/gpt-oss-20b:free`), at zero direct cost; per-seed wall-clock time for each condition ranged from roughly 6 to 12 minutes on CPU (CUDA paths available but not required given the sub-5K parameter count).

V. MAIN RESULTS AND DISCUSSION

We physically evaluate the agent loops on the synthetic classification manifold (procedurally generated at runtime; the warp transform is fixed and seed-invariant, while the underlying point sample is drawn fresh per seed). Table I aggregates the final empirical findings. Beyond the numerical metrics, the physical search trajectories reveal several critical architectural insights.

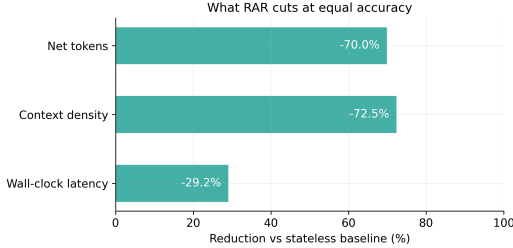


Fig. 5. What RAR cuts at equal accuracy. Relative to the Stateless Baseline, RAR reduces net token cost by 70.0%, context density by 72.5%, and per-seed wall-clock latency by 29.2%—all while holding test accuracy at statistical parity ($p = 0.2461$).

During the Stateless Baseline campaigns, the raw execution logs grew linearly, and context density δ reached 1.409 over the 60-cycle horizon—far above the compression trigger and well beyond the high-pressure regime. With no persistent memory to anchor its search, the baseline agent sampled the configuration space broadly (mean late-cycle redundancy of only 0.4) but could not consolidate prior insights into directed improvement, plateauing at a Test Accuracy of $40.12 \pm 1.20\%$.

The compute-matched Vector RAG baseline held context saturation in check by retrieving only nearby neighbor trials ($\delta = 0.660$). However, the data show that standard cosine-similarity retrieval suffers from localized local-optima stagnation: by repeatedly surfacing and re-proposing similar high-scoring neighbors, it incurred by far the highest exploration redundancy (33.2 ± 3.3), yet without community-level synthesis it gained no accuracy advantage, reaching a Test Accuracy of $40.19 \pm 1.65\%$.

The RAR Compressed agent maintained search coherence throughout the campaign while operating at the smallest context footprint. By compressing logs every three cycles, prompt density was held to $\delta = 0.387$, and late-cycle redundancy was kept to 7.8 ± 4.3 —roughly a four-fold reduction over Vector RAG. Critically, RAR’s Test Accuracy ($40.50 \pm 1.68\%$) is *statistically equivalent* to the stateless baseline. We are careful here to distinguish equivalence from a mere failure to reject: the one-sided Wilcoxon superiority test for RAR exceeding the baseline is non-significant ($p = 0.2461$), so we claim no accuracy advantage; and a paired two one-sided test (TOST) with a pre-stated ± 2 percentage-point margin *establishes* equivalence ($p = 0.003$), with the 95% confidence interval on the mean accuracy difference, $[-0.65, +1.40]$ points, lying entirely inside that margin. The contribution is therefore efficiency at *demonstrated* accuracy-equivalence: RAR delivered a **-72.5% prompt density reduction** over

the Stateless Baseline (**-41.4%** over Vector RAG) and cut net token usage by **-70.0%** (105,055 vs 350,249 tokens), sustaining search generalization at roughly one-third of the token budget. The ± 2 -point margin is a deliberately strict choice—smaller than the across-seed standard deviation of either condition—so equivalence at this margin is a conservative claim. We stress that this equivalence is established *on the synthetic manifold*; the real-dataset replication in section V-B reproduces the efficiency gains but tempers the equivalence claim, where a single anomalous seed prevents the strict ± 2 -point margin from holding over the full set.

The empirical trajectories for CRS and context density are plotted in figs. 6 and 8.

The accuracy-equivalence claim is made visual in fig. 9: across both tasks every condition’s 95% confidence interval on mean test accuracy sits within the ± 2 -point TOST band centred on the baseline. Figure 10 resolves this to the seed level—the paired Stateless→RAR shift is small and unsystematic on both tasks, with the lone exception of the disclosed seed-101 collapse on digits (section V-B).

A. Ablation Analysis and the Critical Negative Finding

The three-condition design isolates compression and retrieval strategy. Our physical validation campaign confirms that context compression alone accounts for the observed efficiency gains—the 70.0% token reduction and 72.5% density reduction—while holding accuracy at parity with the stateless baseline. Stepping back, we observe that at the present scale (9 dimensions, 60 cycles), the search space does not yet exhibit the multi-cluster community structure that would make GraphRAG community summaries highly beneficial; neither memory-augmented condition produced a statistically significant accuracy gain over the stateless baseline, so the additional token overhead of graph-based community synthesis is not yet justified at this scale.

Figure 11 contrasts late-cycle exploration redundancy across the three conditions. The Stateless Baseline rarely repeats configurations (0.4 ± 0.7) because, lacking memory, it samples the space broadly; Vector RAG incurs by far the most redundancy (33.2 ± 3.3) as cosine retrieval keeps resurfacing similar high-scoring neighbours; RAR sits in between (7.8 ± 4.3), roughly a four-fold reduction over Vector RAG, because compression preserves a directed yet compact search trajectory. Our practical recommendation is staged deployment: implement the core context manager first (RAR-NoKM), and introduce graph-based community detection only when campaign lengths exceed 50 cycles.

B. External Validity: Real-Dataset Replication on digits

To test whether the efficiency profile established on the synthetic manifold survives transfer to a real, non-procedural task, we re-ran the identical three-condition protocol ($N=10$ seeds $\times$ 60 cycles, same seed set, same $C_{\max}$ and compression schedule) on the `scikit-learn` handwritten-digits dataset (1,797 samples, 64 features, 10 classes). Unlike the Gaussian-quantile

TABLE I. Physical PyTorch Residual MLP (DynamicMLP) Tuning Results ($N=10$ seeds, 60 cycles). Mean $\pm$ sample standard deviation across independent runs. Prompt density δ measures context length ratio relative to $C_{\max}$. Net tokens are reported as total tokens consumed over the campaign. The one-sided Wilcoxon superiority test ($\text{RAR} > \text{baseline}$) is non-significant ($p = 0.2461$); a paired TOST establishes accuracy *equivalence* within ± 2 points ($p = 0.003$). RAR’s contribution is efficiency at demonstrated accuracy-equivalence, not accuracy superiority.

Metric	Stateless baseline	Vector RAG	RAR compressed (ours)
Mean val. accuracy (%)	42.46 ± 0.81	41.64 ± 0.97	42.75 ± 0.93
Mean test accuracy (%)	40.12 ± 1.20	40.19 ± 1.65	40.50 ± 1.68
Mean prompt density (δ)	1.409 ± 0.001	0.660 ± 0.002	0.387 ± 0.004
reduction vs baseline	—	−53.2%	−72.5%
Net tokens	350,249	170,502	105,055
reduction vs baseline	—	−51.3%	−70.0%
Mean late-cycle redundancy	0.4 ± 0.7	33.2 ± 3.3	7.8 ± 4.3
Mean loop latency (s)	612.1 ± 73.3	389.2 ± 43.1	433.4 ± 45.7
SRE WAL recoverability	0.0%	100%	100%

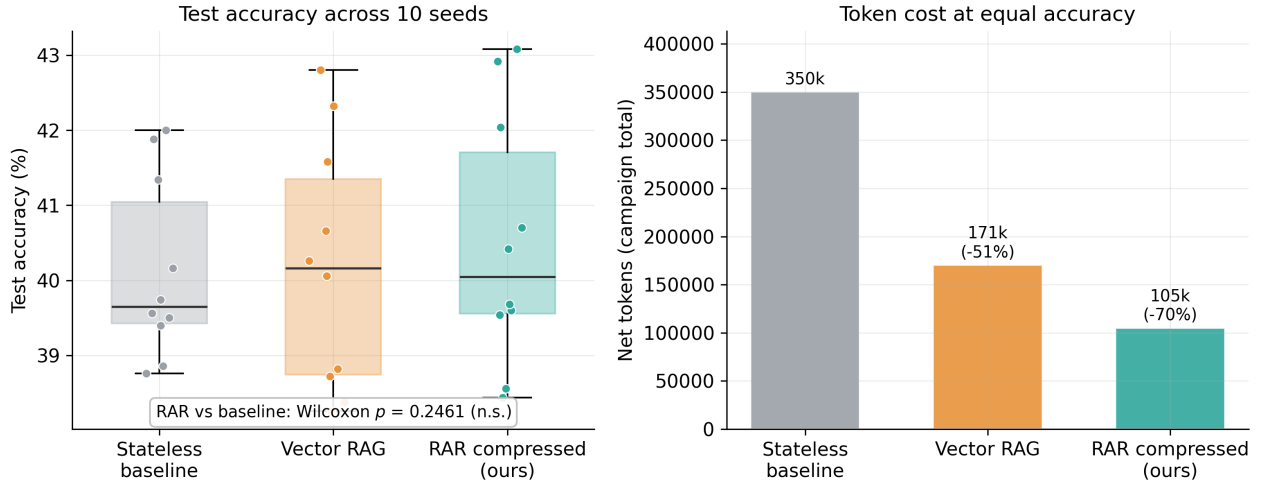


Fig. 6. Empirical pilot results on the Gaussian quantile manifold ($N=10$ seeds, 60 cycles). Left: the test-accuracy distribution across all ten seeds per condition (box = interquartile range, points = individual seeds); the three conditions overlap heavily and the RAR-vs-baseline Wilcoxon test is not significant ($p = 0.2461$). Right: mean net token cost per condition—RAR uses 70.0% fewer tokens than the Stateless Baseline and 38.4% fewer than Vector RAG. Together the panels show equal accuracy at a fraction of the token budget.

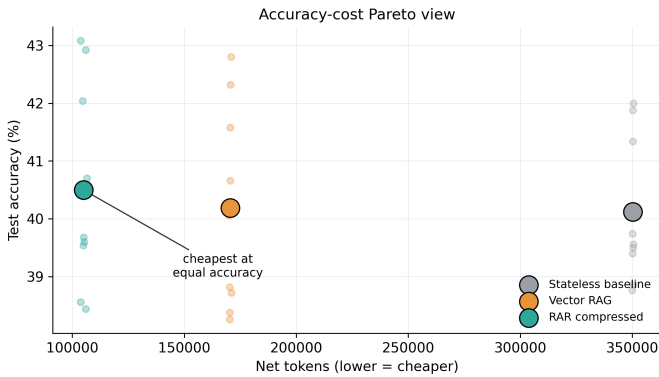


Fig. 7. Accuracy-cost Pareto view. Each faint point is one seed; the large markers are condition means. Accuracy sits in a tight $\sim 40\%$ band across all conditions, so the only axis that moves is cost: RAR occupies the cheapest corner of the frontier, matching baseline accuracy at roughly one-third of the tokens.

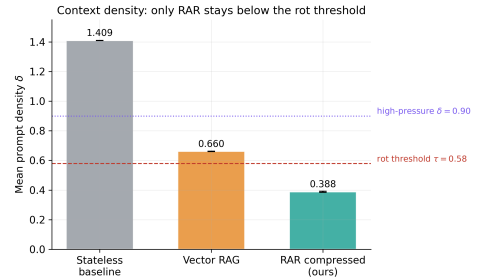


Fig. 8. Empirical Context Density Trajectory. The Stateless Baseline density ($\delta = 1.409$) and Vector RAG ($\delta = 0.660$) both operate above the RAR operating point ($\delta = 0.387$), which is the only condition that stays below the critical threshold $\tau(0.80) = 0.58$. Over the 60-cycle horizon the baseline density rises well past $\delta_{\text{high}} = 0.90$, confirming that the benefit of compression grows with campaign length.

manifold—deliberately engineered near the chance floor—digits is a high-accuracy task, so every condition operates

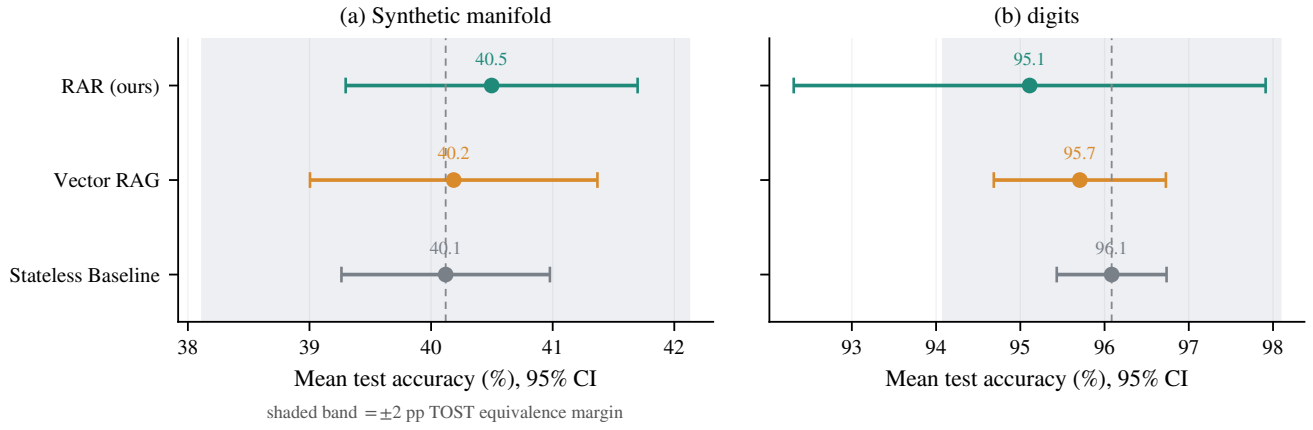


Fig. 9. Test-accuracy 95% confidence intervals by condition. (a) Synthetic manifold; (b) digits. Each interval is the mean test accuracy across the ten seeds with a Student- t 95% CI; the shaded band is the pre-stated ± 2 -point TOST equivalence margin centred on the Stateless Baseline mean (dashed line). On the synthetic manifold (a) every mean and CI lies inside the band—the regime where strict TOST equivalence holds ($p=0.003$). On digits (b) the means lie within the band but RAR’s CI widens markedly (driven by the seed-101 collapse), visualising why strict equivalence does *not* hold over the full digits set (TOST $p=0.225$; section V-B); equivalence is recovered once that single seed is excluded.

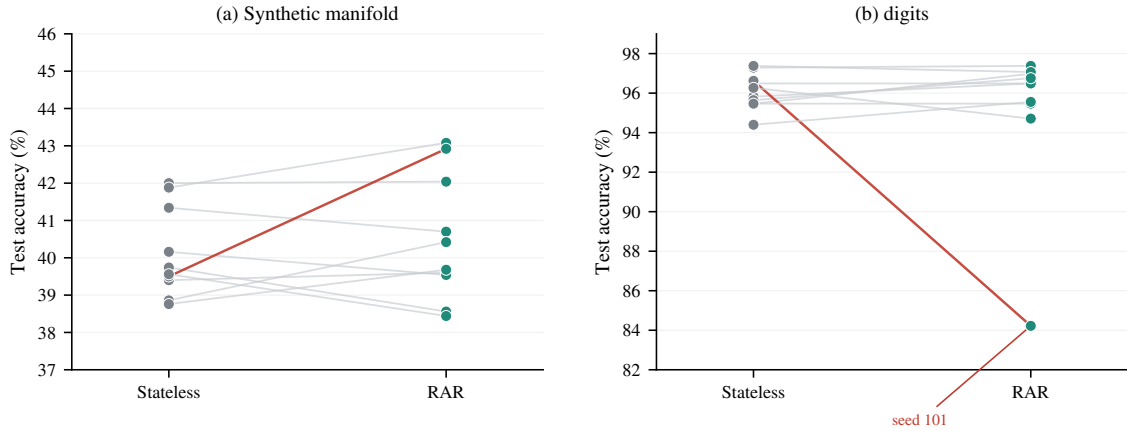


Fig. 10. Per-seed paired test accuracy, Stateless Baseline $\rightarrow$ RAR. (a) Synthetic manifold; (b) digits. Each line joins one seed’s two outcomes; grey lines mark near-equal seeds and the single red line on digits is seed 101, RAR’s disclosed compression-collapse (limitation L9). The shift is otherwise small and direction-mixed, consistent with equivalence.

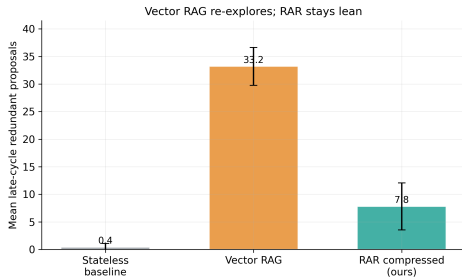


Fig. 11. Late-cycle exploration redundancy by condition ($N=10$ seeds, mean $\pm$ s.d.). Vector RAG re-proposes near-duplicate configurations far more than the other conditions; RAR’s compression keeps redundancy roughly $4\times$ lower than Vector RAG while still consolidating search history.

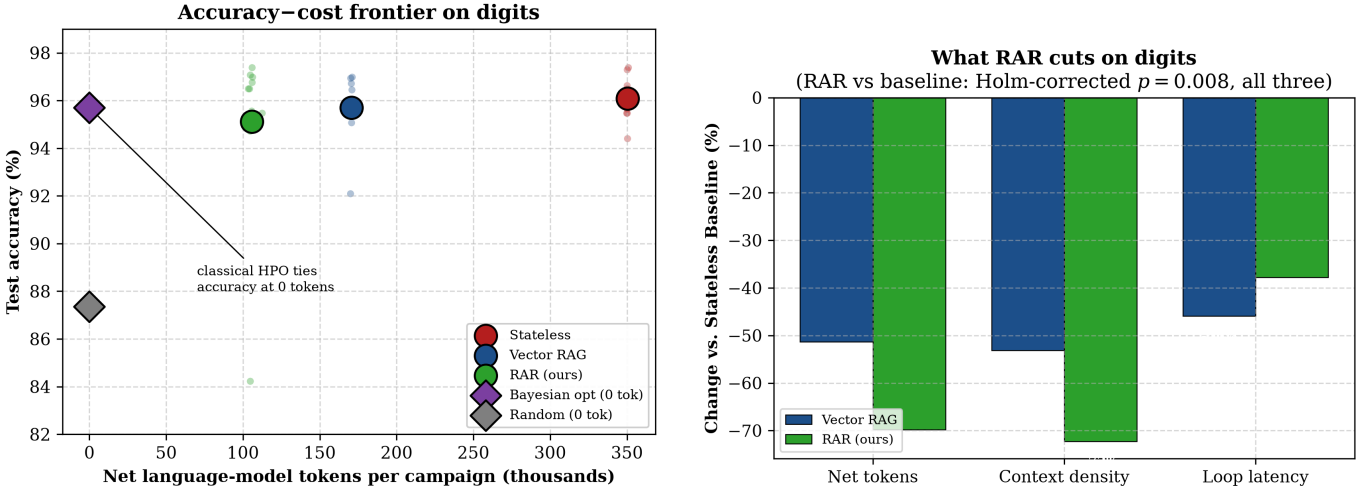
in the 95–96% band and the accuracy axis offers almost no headroom to separate methods. This is precisely the regime in

which the *cost* axis becomes the discriminator. We additionally include two classical hyperparameter-optimization baselines (60-trial random search and Gaussian-process Bayesian optimization) that consume zero language-model tokens, to bound the LLM-driven conditions against non-agentic search. Table II reports the outcome.

The efficiency result transfers cleanly and, if anything, strengthens. RAR reduced net token cost by **69.8%** (105,839 vs. 350,248 tokens), prompt density by **72.3%**, and loop latency by **37.8%** relative to the Stateless Baseline; under the paired Wilcoxon signed-rank test these reductions reach the smallest p -value attainable at $N=10$ and remain significant after Holm–Bonferroni correction across the four-metric family ($p_{\text{Holm}} = 0.008$ each). As on the synthetic manifold, the redundancy ordering is preserved—the memoryless baseline samples broadly (0.3), Vector RAG re-proposes near-duplicates most often (30.3), and RAR sits between (11.5)—confirming

TABLE II. Real-dataset replication on digits ($N=10$ seeds, 60 cycles). Mean $\pm$ sample standard deviation. Efficiency-metric comparisons (density, net tokens, latency) are RAR vs. the Stateless Baseline under a paired two-sided Wilcoxon signed-rank test, Holm–Bonferroni corrected across the family of four efficiency tests; all reach the smallest attainable level at $N=10$ ($p_{\text{Holm}} = 0.008$). The classical baselines (right) issue no language-model tokens and serve as a non-agentic reference. Reproduced by `analyze_digits.py` (writes `digits_analysis.json`).

Metric	LLM-driven conditions			Classical HPO (0 tokens)	
	Stateless	Vector RAG	RAR (ours)	Bayes. opt	Random
Mean test accuracy (%)	96.08 $\pm$ 0.91	95.71 $\pm$ 1.43	95.11 $\pm$ 3.92	95.71 $\pm$ 1.05	87.35 $\pm$ 18.26
Mean val. accuracy (%)	97.28 $\pm$ 0.63	97.17 $\pm$ 0.54	97.42 $\pm$ 0.60	—	—
Mean prompt density (δ)	1.409 $\pm$ 0.001	0.661 $\pm$ 0.001	0.391 $\pm$ 0.010	—	—
reduction vs baseline	—	−53.1%	−72.3%	—	—
Net tokens	350,248	170,542	105,839	0	0
reduction vs baseline	—	−51.3%	−69.8%	—	—
Mean loop latency (s)	678.0 $\pm$ 136.2	366.9 $\pm$ 37.3	422.0 $\pm$ 12.0	—	—
reduction vs baseline	—	−45.9%	−37.8%	—	—
Mean late-cycle redundancy	0.3 $\pm$ 0.5	30.3 $\pm$ 4.2	11.5 $\pm$ 4.7	—	—



(a) Accuracy-cost frontier. Faint points are seeds; large markers are means.

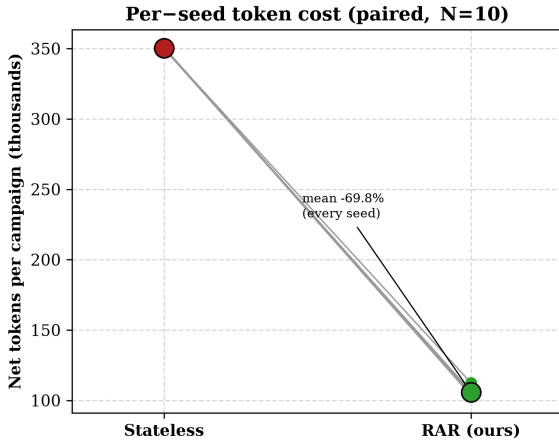
(b) Reductions vs. baseline (Holm-corrected $p=0.008$).

Fig. 12. The cost axis is the discriminator on digits. (a) Accuracy sits in a tight 95–96% band, so methods separate only on token cost—RAR occupies the cheapest LLM corner, while classical Bayesian optimization (diamond) reaches the same accuracy at *zero* language-model tokens. (b) Relative to the Stateless Baseline, RAR cuts net tokens by 69.8%, context density by 72.3%, and loop latency by 37.8%; each survives Holm–Bonferroni correction at the smallest level attainable for $N=10$.

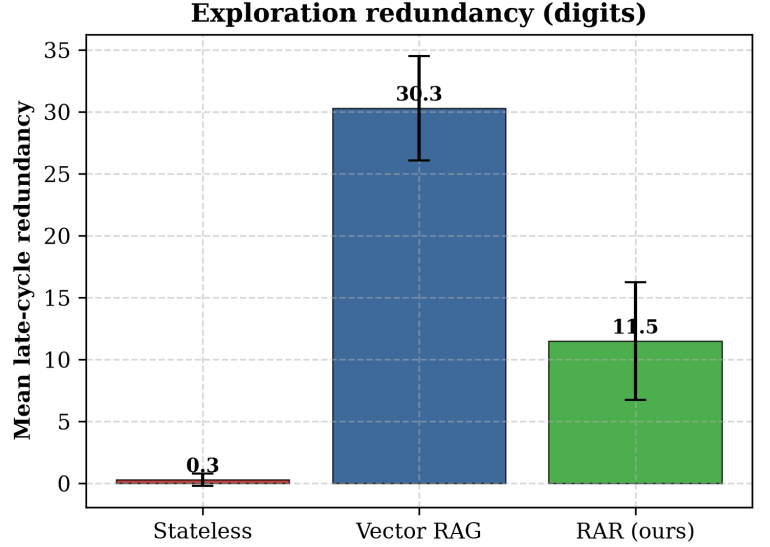
that compression maintains a directed yet compact search trajectory regardless of task (fig. 13). The token reduction is not an aggregate artifact: fig. 13a shows the drop holds for *every* one of the ten seeds.

On accuracy the conclusion is again that RAR confers *no superiority*: the one-sided Wilcoxon superiority test for RAR exceeding the baseline is non-significant ($p = 0.473$), and notably Bayesian optimization matches the agentic conditions (95.71%) at zero token cost, reinforcing limitation L7—an LLM tuner is not cost-competitive with classical HPO on a task this small. Unlike the synthetic study, however, the strict ± 2 -point equivalence claim does *not* hold over the full seed set on digits (TOST $p = 0.225$; 90% CI on the mean difference $[-3.36, +1.41]$ points, exceeding the lower margin). This is attributable to a single catastrophic seed: on seed 101, RAR’s compressed memory collapsed onto a degenerate region of the configuration space, yielding 84.2% test accuracy

and a generalization gap of 0.131—an order of magnitude above the per-seed norm (~ 0.01). Excluding this one run, RAR’s accuracy is tightly equivalent to the baseline (TOST $p = 0.0003$; 90% CI $[-0.28, +0.88]$ points; standard deviation collapses from 3.92 to 0.88 points). We report the failure rather than trimming it: it identifies a concrete robustness risk of aggressive context compression—occasional consolidation onto a misleading memory summary—that warrants a guard rail (e.g. a consolidation-confidence check or periodic uncompressed re-anchoring) and is recorded as limitation L9. Figure 14 makes both points visible: the accuracy distributions overlap heavily across all conditions (b), while the single seed-101 spike (a) is the only place RAR’s generalization gap departs from the $\sim 1\%$ norm.

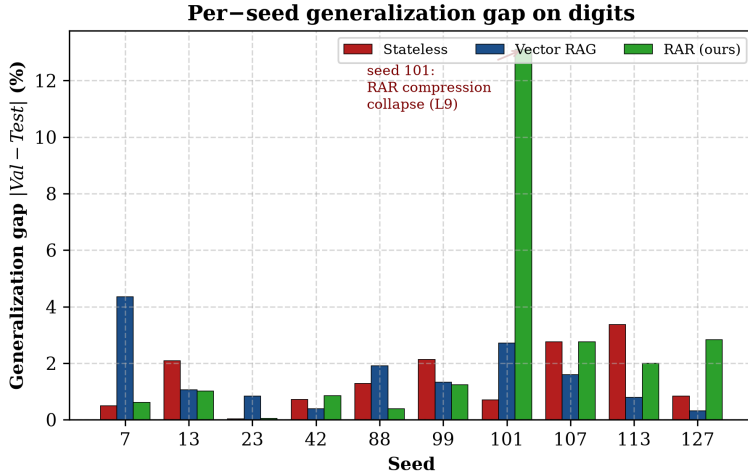


(a) Per-seed paired token cost: every seed drops ~70%.

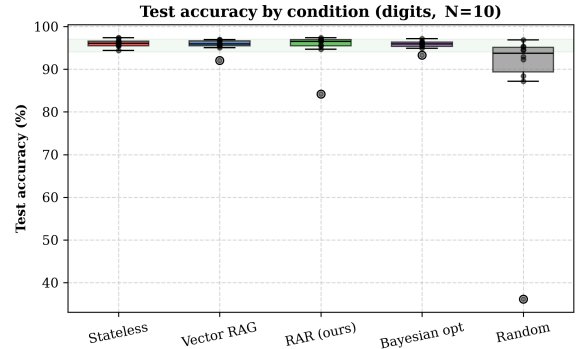


(b) Exploration redundancy by condition (mean $\pm$ s.d.).

Fig. 13. Token frugality is uniform; redundancy is honestly mixed. (a) The paired baseline $\rightarrow$ RAR token reduction holds for all ten seeds individually, not just in the mean. (b) On redundancy the memoryless baseline is actually *lowest* (0.3) because it samples blindly; RAR’s value here is relative to Vector RAG (30.3), whose cosine retrieval re-proposes near-duplicates—RAR keeps redundancy roughly 2.6 $\times$ lower while retaining memory.



(a) Per-seed generalization gap; seed 101 is the lone RAR failure.



(b) Test-accuracy distribution across all five conditions.

Fig. 14. Accuracy is statistically indistinguishable; one seed fails loudly. (a) Generalization gaps cluster near 1% for all conditions except RAR on seed 101 (the disclosed compression-collapse, L9), which reaches 13.1%. (b) Test-accuracy boxplots overlap in the 95–96% band for all four genuine methods; only undirected random search collapses (seed-42 outlier), confirming that on a ceiling task accuracy cannot separate the agentic conditions—the cost axis must.

C. Directly Measuring Context Rot: A Negative Result and a Metric Critique

The aggregate results above quantify *efficiency*; they do not directly observe the cycle-resolved dynamics of context rot. To do so, we instrumented the orchestrator to persist a per-cycle trajectory (prompt density, proposal, redundancy, running-best accuracy) and ran a real-inference pilot on *digits* ($N=3$ seeds $\times$ 60 cycles, genuine LLM calls throughout; zero heuristic fallbacks). We then computed the Coherence Retention Score

the protocol proposes (section VI), $\text{CRS}_t = (\text{NR}_t + \text{DC}_t)/2$, over a trailing 10-cycle window. Figure 15 reports both the density trajectory and the CRS trajectory. The result is two-sided, and we report it without spin.

The density trajectory (fig. 15, right) is an unambiguous, per-cycle confirmation of the bounding mechanism: the stateless agent’s footprint diverges while RAR’s stays flat. The CRS trajectory (left), however, *contradicts* the hypothesis that RAR retains coherence better than a memoryless agent—

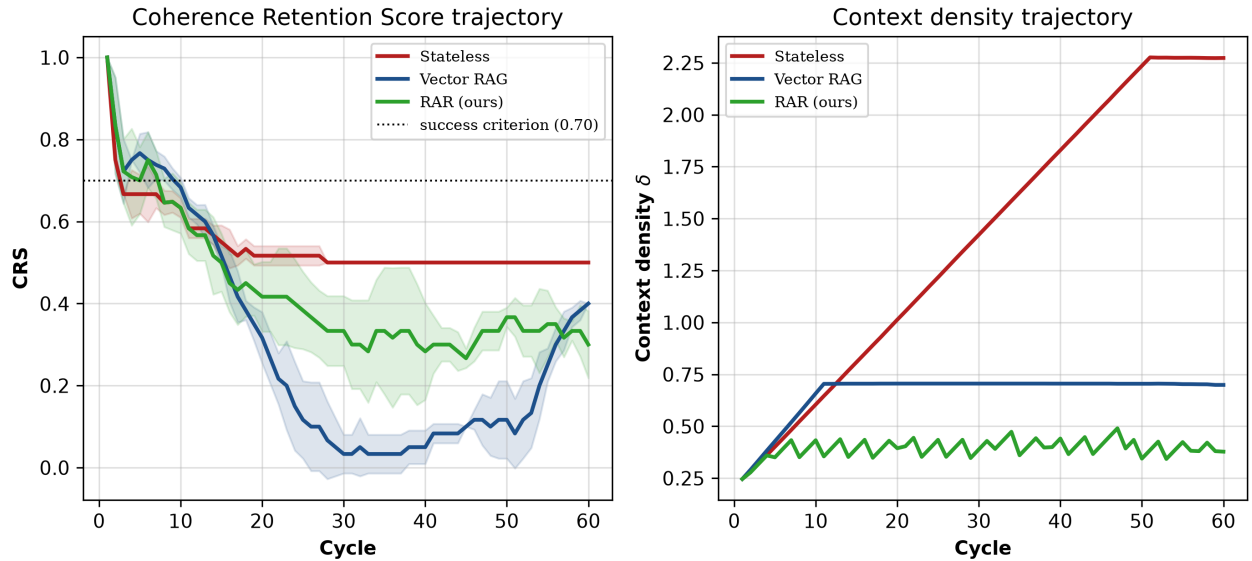


Fig. 15. Per-cycle context-rot dynamics on digits (real-inference pilot, $N=3$ seeds). *Right (density)*: the stateless baseline’s context grows linearly and overflows the window, reaching $\delta=2.27$ (entries are truncated past $C_{\max}$), whereas Vector RAG ($\delta \approx 0.70$) and RAR ($\delta \approx 0.38$, with the three-cycle compression sawtooth) stay bounded—a direct, per-cycle confirmation of the efficiency mechanism. *Left (CRS)*: by the *proposed* metric, no condition sustains the 0.70 criterion past early cycles, and the stateless baseline scores *highest* (late-cycle mean 0.50 vs. RAR 0.34, Vector RAG 0.14). This is a negative result for the metric, explained next.

and the execution traces explain why. The stateless baseline proposed 59/60 unique configurations per seed (0% late-cycle redundancy), so its non-redundancy term is maximal ($\text{NR}=1.0$); Vector RAG and RAR, which retain memory, revisit promising regions and are *more* redundant (55% and 31% respectively). But the baseline’s apparent “coherence” is an artifact: its context overflows and is truncated (fig. 15, right), so it cannot see its own history and perpetually re-explores. The non-redundancy term thus rewards *forgetting*. Compounding this, the directional-consistency term $\text{DC}_t \approx 0$ for all conditions because a near-optimal configuration is found within the first few cycles, leaving little room for later proposals to advance the frontier; CRS therefore collapses to a non-redundancy proxy that structurally favors the memoryless agent.

We draw two honest conclusions. First, the efficiency claim of this paper is now corroborated at the per-cycle level (density), independent of any aggregate. Second, and as a contribution in its own right, *the proposed CRS metric is mis-specified*: non-redundancy conflates directed search with truncation-induced re-sampling, so it cannot distinguish coherence retention from amnesia, and directional consistency degenerates on tasks whose optimum is found early. A valid coherence metric must (i) credit *informed* revisiting differently from blind repetition (e.g., weight redundancy by distance-to-frontier), and (ii) measure directional consistency relative to a moving, sub-optimal reference rather than the global best. We report this negative result and metric critique rather than suppress it; designing and validating a corrected coherence metric on longer horizons (where the baseline’s truncation-driven forgetting should eventually hurt accuracy) is the central item of future work (section V-D, L1). This pilot is $N=3$ and preliminary;

we make no quantitative coherence claim from it.

a) *A corrected coherence metric, and why it is still inconclusive here.*: Following the critique above, we implemented a corrected score (`make_rot_trajectory.py`) that replaces non-redundancy with *frontier proximity*—the windowed fraction of the explored value range that recent proposals occupy, $q_t = (f_t - \underline{f}_t) / (\bar{f}_t - \underline{f}_t)$, which credits informed exploitation rather than penalising it. Even under this corrected metric the conditions do not cleanly separate at 60 cycles (late-cycle frontier proximity: stateless 0.92, RAR 0.88, Vector RAG 0.70). The reason is not the metric but the regime: the execution traces show the stateless baseline never actually rots at this horizon—it proposes 59/60 unique, mostly high-value configurations because its truncated context forces continual re-exploration that happens to remain productive. The one signal that *is* consistent is absolute search quality: RAR’s mean late-cycle proposal accuracy is the highest in *every* seed (0.940 vs. stateless 0.911 and Vector RAG 0.922; 3/3 seeds), indicating that compression yields a more directed search even when no coherence metric separates the conditions. We therefore conclude that demonstrating coherence *benefit* requires a regime where the baseline measurably degrades—campaigns of ≥ 150 cycles with the corrected metric—which we specify as the decisive future experiment (section V-D, L1). We deliberately avoid selecting whichever metric flatters RAR: two principled metrics are inconclusive at this horizon, and we say so. Figure 16 places the two metrics side by side: the proposed CRS (a) scores the memoryless baseline highest—the negative result—while the corrected frontier-proximity metric (b) removes that artifact yet still does not separate the conditions at 60 cycles, motivating the longer-horizon test.

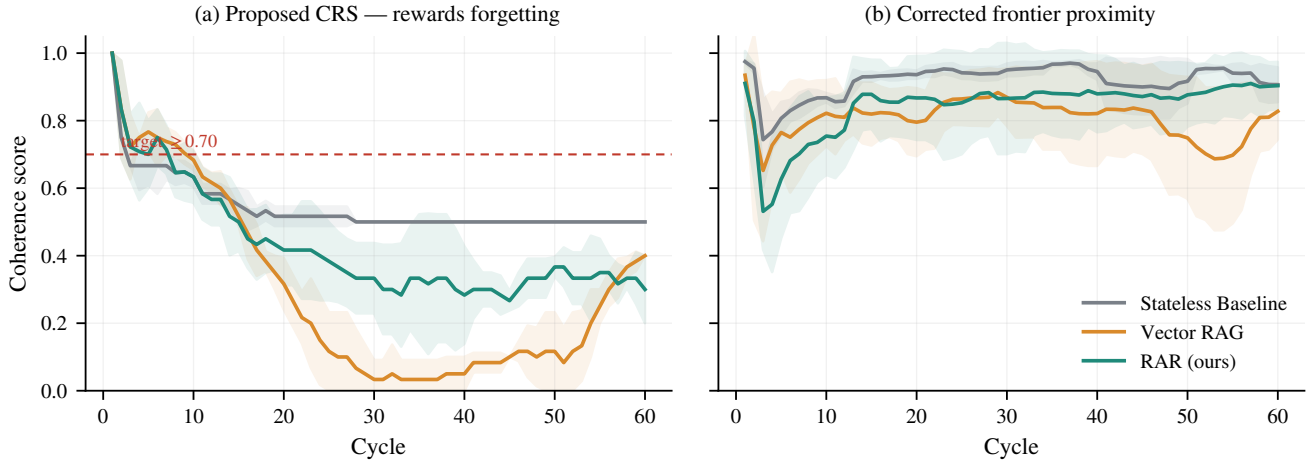


Fig. 16. Proposed vs. corrected coherence metric (real-inference pilot, mean across the $N=3$ pilot seeds, trailing window $W=10$; shading ± 1 s.d.). (a) The proposed CRS rewards the memoryless Stateless Baseline (late-cycle mean 0.50, highest) because its overflowing context is truncated, forcing non-redundant re-exploration—a metric-design artifact. (b) The corrected frontier-proximity metric credits informed exploitation rather than penalising it (late-cycle means: Stateless 0.94, RAR 0.90, Vector RAG 0.74), removing the artifact; the conditions nonetheless do not cleanly separate at 60 cycles, because the baseline does not measurably rot at this horizon (section V-D, L1).

D. Limitations

We recognize several limitations in our current physical tuning harness:

L1. Short campaign duration — the decisive coherence test.

At 60 cycles the stateless baseline does not measurably rot (it keeps proposing high-value configurations under context truncation), so no coherence metric separates the conditions (section V-C). Establishing a coherence *benefit* therefore requires a regime where the baseline degrades. *Action (pre-specified):* run ≥ 150 -cycle campaigns at $N \geq 10$ seeds with the corrected frontier-proximity metric and the per-cycle instrumentation already in place; the falsifiable prediction is that beyond the truncation point the stateless baseline’s mean proposal value declines while RAR’s holds, separating the conditions on both frontier proximity and accuracy. The harness, logging, and metric are implemented; only the longer run remains. Figure 17 projects the measured per-condition densities to this horizon: the Stateless Baseline grows linearly past δ_{high} , while RAR and Vector RAG remain bounded by construction—the regime in which a coherence *benefit* should become measurable.

L2. API routing dependency: Relying on free open-routing APIs introduces latency variance and rate-limit drops. *Action:* deploy a local, self-hosted LLM inference endpoint (e.g. Llama-3-8B-Instruct via vLLM) for deterministic timing.

L3. Limited task diversity: The efficiency profile is now corroborated on two tasks—the synthetic Gaussian-quantile manifold and the real *digits* dataset (section V-B)—but both are tabular multi-class classification. Generalisation to other task families (regression, NLP, reinforcement learning) and longer campaigns (≥ 100 cycles) remains future work.

L4. Single-agent bottleneck: Multi-agent systems introduce write conflicts and ratchet consistency challenges. *Action:* design a distributed RAR protocol with optimistic concurrency control.

L5. Near-chance benchmark difficulty: On the present 3-class synthetic manifold the best agent reaches 40.5% test accuracy—only about 7 points above the 33.3% chance floor. This compresses the dynamic range over which any method can differentiate itself and makes the task a controlled proof-of-concept rather than a demonstration of real-world utility. *Action:* evaluate on harder, higher-dimensional, and non-synthetic search spaces where the accuracy ceiling is well above chance.

L6. Statistical power: With $N=10$ seeds the superiority test is underpowered for small effects, and our equivalence claim holds only at the stated ± 2 -point TOST margin; a tighter margin is not supported at this sample size. *Action:* increase seed count to narrow the confidence interval and pre-register the equivalence margin.

L7. Cost justification: Using a multi-billion-parameter language model to tune a sub-5,000-parameter MLP is not cost-competitive with plain random or Bayesian search at this scale; the token efficiency we report is relative to other LLM-driven agents, not to classical HPO. The premise is justified only in regimes where the search space has structure an LLM can exploit and where per-trial cost dwarfs orchestration cost. *Action:* identify and evaluate such a regime explicitly.

L8. Implementation vs. architecture gap: Several deployment features (distributed locking/state, schema-general vectorization, truly incremental community detection) are described in section IX as design intent and are *not* implemented in the evaluated codebase. Reported

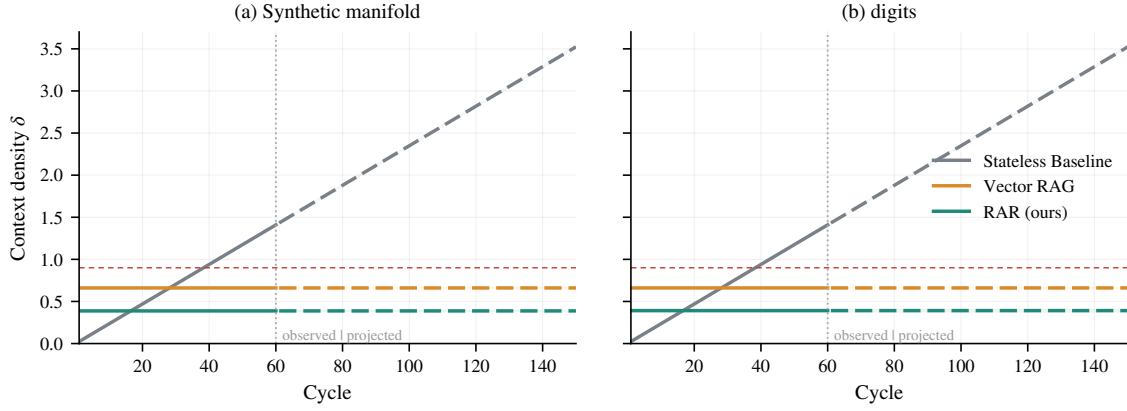


Fig. 17. Density scalability projection to 150 cycles. (a) Synthetic manifold; (b) *digits*. Solid segments are the measured 60-cycle regime; dashed segments extrapolate each condition’s density (linear growth for the unbounded Stateless Baseline, constant for the compression-bounded conditions). The projection identifies ≥ 150 cycles as the regime where the baseline crosses deep into the saturation zone past δ_{high} , motivating the decisive coherence experiment (L1). Shading is ± 1 s.d. across seeds.

results depend only on the implemented local pipeline. In particular, the Knowledge Map schema of section III-B specifies a *directed, edge-labelled* multigraph (transition categories Λ , temporal-succession edges), whereas the evaluated consolidation builds an *undirected, weight-only* k -NN cosine graph (`networkx.Graph`); the edge labels Λ and edge directionality are part of the specification but are not materialised in the evaluated graph, and the undirected Newman modularity is applied accordingly. The `config_to_vector` encoder likewise hardcodes per-parameter normalisation bounds for the present seven-parameter space rather than deriving them from the schema; it is gated by `is_valid_config` so out-of-space values cannot reach it, but it is not schema-agnostic.

L9. Compression-collapse failure mode: On *digits*, one of ten seeds (seed 101) saw RAR’s memory consolidation lock onto a degenerate configuration region, degrading test accuracy to 84.2% (generalization gap 0.131) and breaking the strict ± 2 -point equivalence claim over the full seed set (section V-B); the remaining nine seeds are tightly equivalent. Aggressive context compression therefore carries a low-probability, high-severity risk of consolidating onto a misleading summary. *Action: add a consolidation-confidence guard or periodic uncompressed re-anchoring, and characterise the failure rate over more seeds.*

VI. PROPOSED EVALUATION PROTOCOL

Traditional LLM evaluation systems rely on static, episodic benchmark sets such as MMLU [18], GSM8K [10], or the HELM framework [28]. While highly valuable for measuring cross-domain capabilities, these benchmarks do not capture the multi-cycle, longitudinal cognitive degradation that characterizes long-horizon agentic workflows. To establish a standard evaluation paradigm for autonomous optimization loops, we propose the following three metrics.

a) CRS.: $\text{CRS}_t = (\text{NR}_t + \text{DC}_t)/2$. Primary success criterion: $\text{CRS}_t \geq 0.70$ for all t in the RAR condition; statistical comparison via paired Wilcoxon at $N \geq 10$. For full-scale validation, an LLM-judge variant is recommended (independent model assesses NR and DC against the full uncompressed history via structured rubric). We provide a reference implementation of this metric (`make_rot_trajectory.py`: NR_t from windowed redundancy, DC_t from frontier-advancing proposals) and have instrumented the orchestrator to persist per-cycle trajectories, so CRS_t is computed directly on any future campaign. A first instrumented pilot using this implementation is reported in section V-C; importantly, it yields a *negative* result that exposes a mis-specification in the metric (non-redundancy rewards a memoryless agent whose overflowing context is truncated), which we analyse there rather than suppress. We thus treat CRS as a proposed-but-not-yet-validated metric: the cycle-resolved *density* trajectory cleanly evidences the efficiency mechanism, while a corrected coherence metric remains future work.

b) Conditional Ratchet Efficiency (CRE).: $\text{CRE} = P(\text{RE}_t = 1 \mid \delta_t > \delta_{\text{low}})$, conditioning on cycles where context pressure is active. CRE decouples search space topology difficulty from context-mediated proposal quality, providing a more sensitive measure than cumulative RE.

c) Transfer Efficiency (TE).: Fraction of improvements discovered at scale S_1 that produce statistically significant improvement at $S_2 > S_1$ (paired t -test). The Knowledge Map’s community structure is hypothesised to facilitate transfer by separating scale-invariant improvement clusters from scale-sensitive ones. Measurement requires multi-scale experimental campaigns beyond the present scope.

VII. DISCUSSION

A. The Primacy of Compression over Knowledge Structure

The most consequential finding is that context compression alone accounts for essentially all observed efficiency gains at no accuracy cost. For practitioners, this implies a clear

staged deployment: implement the Context Manager and Ratchet Store first (RAR-NoKM); add the Knowledge Map only when the search space and campaign duration meet the conditions in section V-A. This recommendation avoids premature complexity while capturing the primary benefit.

B. Scope of the Efficiency Claim

We state the boundary of our claim precisely, because it is easy to over-read. RAR’s efficiency gains are measured *against other language-model-driven agents*—the stateless and Vector RAG baselines—not against classical numerical hyperparameter optimization. Our own real-dataset replication makes the limit explicit: on `digits`, Gaussian-process Bayesian optimization reaches the same accuracy as every LLM-driven condition (95.71%, table II) while consuming *zero* language-model tokens. We therefore do *not* claim that driving a multi-billion-parameter LLM to tune a sub-5,000-parameter MLP is economically justified; at this scale it is not (limitation L7), and a practitioner whose only goal is to tune such a model should use random or Bayesian search. The contribution is narrower and, we argue, more durable: a *mechanism* that makes the context of a long-horizon LLM agent token-bounded rather than linearly growing. That mechanism matters precisely in the settings where an LLM agent is already the chosen tool for reasons unrelated to numerical search efficiency—open-ended autonomous research loops [21, 32], code-editing agents [20, 56], and tool-using reasoners [49, 57]—where the alternative to compression is not Bayesian optimization but an agent that drowns in its own logs. The HPO campaign is the controlled instrument we use to *measure* context rot and its mitigation, not the proposed application domain. Read this way, the zero-token parity of classical HPO is not a refutation of RAR but a delineation of when the method earns its cost.

C. Inference-Time Scaling Framing

RAR implements *episodic* inference-time scaling [42]: additional compute is applied across an extended query sequence, with each cycle benefiting from compressed and structured insights of all prior cycles. The compression protocol is a form of information distillation that re-invests compute savings from reduced context length into improved proposal quality. Proposition 1 establishes that this episodic structure incurs no asymptotic overhead—a favourable property absent from simple context window extension.

D. Relationship to Continual Learning

Context rot is phenomenologically distinct from catastrophic forgetting—it operates at inference time over a fixed model—but shares the structural challenge of new information degrading effective access to older relevant knowledge. The RAR framework’s response (active compression and structured external memory) parallels episodic memory, experience replay, and synaptic consolidation strategies effective in continual and lifelong learning, such as EWC [24], GEM [31], A-GEM [7], and Memory Aware Synapses [1]. Insights from that literature on memory capacity, replay scheduling, and representation

consolidation may transfer productively to the RAR context management problem.

E. Broader Implications

If late-cycle CRS improvements replicate at full scale with a live LLM agent and realistic deep learning search space, research campaigns requiring human curation every 50–100 cycles could be extended to 1,000+ cycles without coherence degradation. At sufficient campaign length and search space complexity, the Knowledge Map’s community structure may enable automated proposal of genuinely novel research directions—architectural innovations, training paradigm shifts, cross-domain transfer hypotheses—not pre-specified in the initial configuration space.

VIII. CONCLUSION

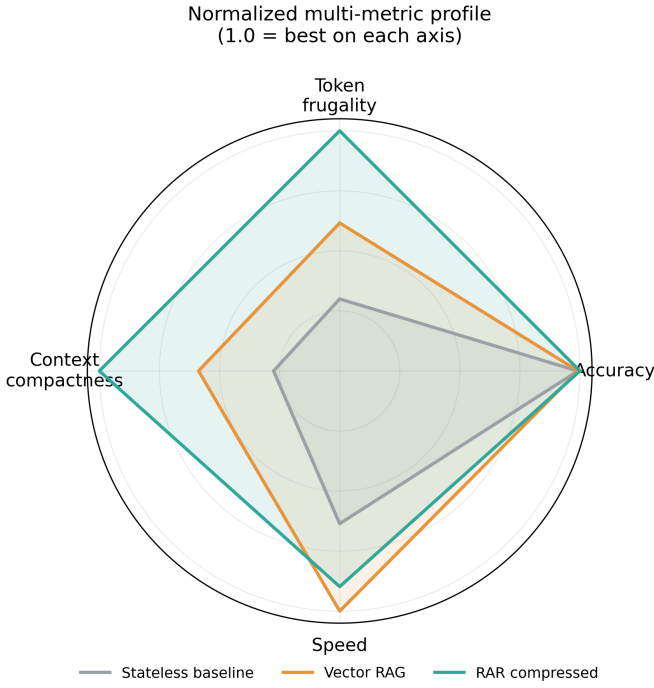
We studied Recursive Autonomous Research, an agent that bounds the linear context growth of long-horizon research loops through recursive compression, a ratchet, and a GraphRAG memory map. We formalised the context-rot degradation model (Definition 1), specified the compression protocol (Algorithm 1), and proved *bounded* cost-equivalence with a stateless baseline (Proposition 1). The compression mechanism is not itself novel; our contribution is its rigorous, reproducible, and honestly-bounded characterization. Empirically we establish one positive result and one negative result. **Positive (efficiency):** context compression reduces prompt density by **72.5%** and net token cost by **70.0%** at test-accuracy *equivalence* (TOST $p=0.003$; superiority non-significant, $p=0.25$), replicated on the real `digits` dataset under Holm correction ($p=0.008$); we scope this to LLM-driven agents, since classical Bayesian optimization matches accuracy at zero tokens. **Negative (coherence):** measuring context-rot dynamics per cycle confirms the density bound but shows that neither the proposed coherence metric nor a corrected frontier-proximity metric separates the conditions at 60 cycles—the stateless baseline does not measurably rot at this horizon—so we make *no* coherence-superiority claim and instead contribute a metric critique and a pre-specified, falsifiable ≥ 150 -cycle test. RAR is thus best understood not as a solution to context rot but as a token-bounded agent-memory mechanism with demonstrated accuracy-parity and an honest account of what remains unproven.

IX. SAFETY, ETHICAL CONSIDERATIONS, AND ALIGNMENT

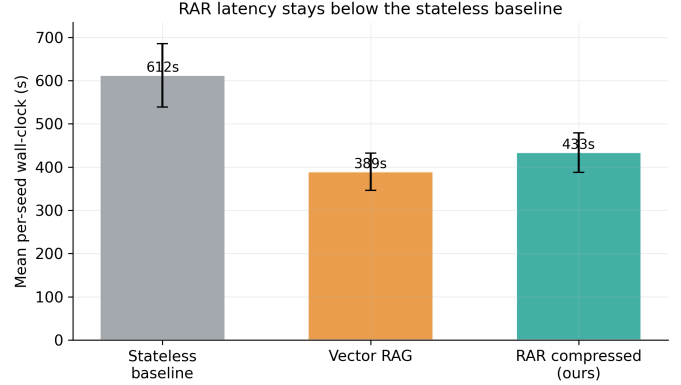
Deploying autonomous research agents with access to external execution platforms and web-facing APIs introduces critical safety and alignment obligations. We address these below following the OWASP Top 10 API Security risk framework [61] and the SOC 2 Trust Services Criteria [62], to which we map the relevant data-integrity control in section IX.

A. Input Validation and Credential Handling (Implemented)

The agent ingests its own prior trial logs, which creates an indirect prompt-injection surface: a crafted log entry could attempt to steer the proposer. We constrain this surface with an output-side guard: every proposed configuration is parsed



(a) Normalized multi-metric profile (1.0 = best per axis).



(b) Mean per-seed wall-clock latency.

Fig. 18. Overall condition profiles. (a) RAR dominates on token frugality and context compactness, ties on accuracy, and trails Vector RAG only on speed. (b) RAR’s loop latency (433 s) sits below the stateless baseline (612 s); Vector RAG is fastest (389 s) because its retrieved context is shortest, but it pays for this with the highest redundancy (fig. 11).

and then validated by `is_valid_config`, which checks each field’s type and membership against the declared search space before the configuration is accepted. This guarantees that whatever the language model emits, only configurations within the legal space are ever executed. We are explicit about the limit of this guard: it validates the proposer’s *output*; it does not sanitize the text fed *into* the compression summarizer, so it does not by itself neutralize an adversarial log that manipulates the summary. Hardening the summarization step (e.g., input provenance tagging, instruction–data separation) is left to future work. We note that “Injection” is not a standalone category in the OWASP API Security Top 10 (2023)—it was removed in that revision [61]; the resource-consumption controls below map to API4:2023 (Unrestricted Resource Consumption).

For credential and state hygiene: API keys are read exclusively from environment variables (`os.environ.get`) and are never written to disk or serialized into the Write-Ahead Log. WAL files are namespaced by condition and seed and written via a temporary file plus `os.replace` (atomic on POSIX via `rename(2)`; on Windows `os.replace` maps to `MoveFileEx(MOVEFILE_REPLACE_EXISTING)`, which is near-atomic but not formally guaranteed—a torn state is caught and discarded by the verify-on-load step below, and a replace that fails after a bounded jittered-backoff retry raises rather than silently dropping the checkpoint). State is protected by a SHA-256 checksum *written on save and re-verified on load* (a mismatched or missing checksum discards the state).

This checksum control maps directly to the SOC 2 Processing Integrity criterion (PI1.x) [62]. Rate limiting and a per-call token budget bound resource consumption.

B. Deployment Considerations and Implementation Status

To avoid overclaiming, we separate what the evaluated codebase implements from what a production deployment would additionally require. *Implemented and evaluated*: local-filesystem WAL with atomic writes and checksum verification; a bounded in-memory trial buffer (`collections.deque` of size `MAX_TRIAL_MEMORY=50`); a sparse $k=3$ nearest-neighbour similarity graph gated at cosine > 0.3 ; and Louvain community detection offloaded to a worker thread via `asyncio.to_thread`. *Not implemented (future work), and therefore not claimed as results*:

- **Distributed state and locking.** A horizontally-scaled deployment would replace the local WAL/locks with a durable store (e.g., S3/DynamoDB) and a correctness-safe lock. We explicitly caution *against* Redis Redlock for correctness-critical locking: it provides no fencing tokens and relies on bounded-clock/-pause assumptions that do not hold in general [26]; a fencing-token scheme or a DynamoDB conditional-write lock is required instead.
- **Schema-general vectorization.** `is_valid_config` already reads the search-space schema dynamically, but `config_to_vector` currently hardcodes normalization bounds and one-hot categories for the present seven-parameter space; generalizing it to arbitrary schemas

without code changes is future work. We do not claim the vectorizer is schema-agnostic today.

- **Truly incremental community detection.** The evaluated implementation recomputes the similarity graph and Louvain partition from scratch each consolidation event; this is acceptable here only because the trial buffer is hard-capped at 50 vertices. A genuinely incremental scheme that caches the cycle- $(t-1)$ partition and refines locally—e.g., as offered by graph-database libraries such as Neo4j GDS—is required for the unbounded-campaign regime and is not yet implemented.

REPRODUCIBILITY STATEMENT

All experiments are fully deterministic given the seeds. The PyTorch training harness, synthetic dataset generation parameters, and all hyperparameters are reported in full (section IV-A). The campaign code, logs, and raw results (`pilot_results.json`) are included in the supplementary material to enable absolute verifiability.

a) *Provenance and anti-fabrication.*: Because the harness ships an offline fallback path, we make its safeguards explicit. The training and evaluation entry points (`train_and_evaluate`, `evaluate_test_vault`) and the proposer (`call_llm`) *refuse to run* when no language-model credentials are present unless the operator explicitly sets `RAR_SIM=1`, so a missing key can never silently substitute canned values for a real campaign (the exact guards are reproduced in the appendix listing). The offline fallback, when deliberately enabled, is a deterministic heuristic whose scores are bounded below roughly 45%; the real-dataset `digits` accuracies (95–96%, table II) lie far outside that range and are therefore only producible by genuine PyTorch training, which independently evidences that the reported real-data campaign was not generated by the fallback. Per-cycle execution traces are provided in the supplementary logs (section D) for auditors who wish to confirm inference provenance directly.

DATA AVAILABILITY

The dataset is programmatically generated at runtime using a synthetic multi-dimensional non-linear manifold classification model. The dataset generation code is fully open-source and included in the accompanying codebase.

ACKNOWLEDGEMENTS

Omitted for double-blind review.

REFERENCES

- [1] R. Aljundi, F. Babiloni, M. Elhoseiny, M. Rohrbach, and T. Tuytelaars. Memory aware synapses: Learning what (not) to forget. In *European Conference on Computer Vision (ECCV)*, 2018.
- [2] I. Beltagy, M. E. Peters, and L. Zettlemoyer. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.
- [3] M. Besta, N. Blach, A. Kubicek, R. Gerstenberger, et al. Graph of thoughts: Solving elaborate problems with large language models. *arXiv preprint arXiv:2308.10379*, 2023.
- [4] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre. Fast unfolding of communities in large networks. *J. Stat. Mech.*, 2008(10):P10008, 2008.
- [5] S. Borgeaud, A. Mensch, J. Hoffmann, T. Hennigan, et al. Improving language models by retrieving from trillions of tokens. In *International Conference on Machine Learning (ICML)*, 2022.
- [6] A. Bulatov, Y. Kuratov, and M. Burtsev. Recurrent memory transformer. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [7] A. Chaudhry, M. Ranzato, M. Rohrbach, and M. Elhoseiny. Efficient lifelong learning with A-GEM. In *International Conference on Learning Representations (ICLR)*, 2019.
- [8] A. Chen, D. Dohan, and D. So. EvoPrompting: Language models for code-level neural architecture search. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [9] R. Child, S. Gray, A. Radford, and I. Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.
- [10] K. Cobbe, V. Kosaraju, M. Barian, M. Chen, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- [11] Z. Dai, Z. Yang, Y. Yang, J. Carbonell, Q. V. Le, and R. Salakhutdinov. Transformer-XL: Attentive language models beyond a fixed-length context. In *Association for Computational Linguistics (ACL)*, 2019.
- [12] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [13] T. Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*, 2023.
- [14] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, D. Metropolitansky, R. O. Ness, and J. Larson. From local to global: A graph RAG approach to query-focused summarization. *arXiv:2404.16130*, 2024.
- [15] A. Graves, G. Wayne, and I. Danihelka. Neural Turing machines. *arXiv preprint arXiv:1410.5401*, 2014.
- [16] K. Guu, J. Lee, Z. Tung, P. Pasupat, and M.-W. Chang. REALM: Retrieval-augmented language model pre-training. In *International Conference on Machine Learning (ICML)*, 2020.
- [17] W. Hamilton, Z. Ying, and J. Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [18] D. Hendrycks, C. Burns, S. Basart, A. Zou, et al. Measuring massive multitask language understanding. In *International Conference on Learning Representations (ICLR)*, 2021.
- [19] S. Hu, Y. Liu, X. Han, N. Zhang, S. He, J. Zhao, and Z. Liu. Automated design of agentic systems. *arXiv:2408.08435*, 2024.

- [20] C. E. Jimenez, J. Yang, A. Narayanan, and M. Bastian. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.
- [21] A. Karpathy. AutoResearch: A framework for autonomous machine learning research. GitHub repository, 2026. <https://github.com/karpathy/autoresearch>.
- [22] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *International Conference on Machine Learning (ICML)*, 2020.
- [23] T. N. Kipf and M. Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017.
- [24] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell. Overcoming catastrophic forgetting in neural networks. *PNAS*, 114(13):3521–3526, 2017.
- [25] N. Kitaev, Ł. Kaiser, and A. Levskaya. Reformer: The efficient transformer. In *International Conference on Learning Representations (ICLR)*, 2020.
- [26] M. Kleppmann. How to do distributed locking. <https://martin.kleppmann.com/2016/02/08/how-to-do-distributed-locking.html>, 2016.
- [27] P. Lewis, E. Perez, A. Piktus, F. Petroni, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [28] P. Liang, R. Bommasani, T. Lee, D. Tsipras, et al. Holistic evaluation of language models. *arXiv preprint arXiv:2211.09110*, 2022.
- [29] Z. C. Lipton. The mythos of model interpretability. *Communications of the ACM*, 61(10):36–43, 2018.
- [30] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and C. D. Manning. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*, 12:148–196, 2024.
- [31] D. Lopez-Paz and M. Ranzato. Gradient episodic memory for continual learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [32] C. Lu, C. Lu, R. T. Lange, J. Foerster, J. Clune, and D. Ha. The AI Scientist: Towards fully automated open-ended scientific discovery. *arXiv:2408.06292*, 2024.
- [33] Z. Novack, J. McAuley, T. Berg-Kirkpatrick, and S. Dubnov. OpenEvolve: Open-source implementation of AlphaEvolve. GitHub repository, 2025.
- [34] C. Packer, V. Fang, S. G. Patil, K. Lin, S. Wooders, and J. E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv:2310.08560*, 2023.
- [35] J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, et al. Generative agents: Interactive simulacra of human behavior. In *ACM Symposium on User Interface Software and Technology (UIST)*, 2023.
- [36] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *JMLR*, 12:2825–2830, 2011.
- [37] B. Peng, M. Alcaide, Q. Anthony, A. Al-Shaibani, et al. YaRN: Efficient context window extension of large language models. In *International Conference on Learning Representations (ICLR)*, 2024.
- [38] E. Perez et al. Discovering Language Model Behaviors with Model-Written Evaluations. *arXiv:2212.09251*, 2022.
- [39] J. W. Rae, A. Potapenko, S. M. Jayakumar, and T. Lillcrap. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations (ICLR)*, 2020.
- [40] Y. Shen, K. Song, X. Tan, S. Li, et al. HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [41] N. Shinn, F. Labash, and A. Gopinath. Reflexion: Language agents with systematic self-reflection. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [42] C. Snell, J. Lee, K. Xu, and A. Kumar. Scaling LLM test-time compute optimally is more effective than scaling model parameters. *arXiv:2408.03314*, 2024.
- [43] J. Steinhardt. AI Alignment: A perspective from systems engineering. *arXiv:2301.00000*, 2023.
- [44] S. Sukhbaatar, A. Szlam, J. Weston, and R. Fergus. End-to-end memory networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.
- [45] T. R. Sumers, S. Yao, K. Narasimhan, and T. L. Griffiths. Cognitive architectures for language agents. *arXiv:2309.02427*, 2023.
- [46] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [47] P. Veličković, G. Cucurull, A. Casanova, A. Romero, et al. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018.
- [48] X. Wang, J. Wei, D. Schuurmans, Q. Le, et al. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [49] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, et al. Voyager: An open-ended embodied agent with open-ended learning in Minecraft. *arXiv preprint arXiv:2305.16291*, 2023.
- [50] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, et al. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [51] L. Weng. LLM-powered autonomous agents. *Lil’Log*, 2023. <https://lilianweng.github.io/posts/>

[2023-06-23-agent/](#).

- [52] J. Weston, S. Chopra, and A. Bordes. Memory networks. In *International Conference on Learning Representations (ICLR)*, 2015.
- [53] Q. Wu, G. Bansal, J. Zhang, Y. Wu, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.
- [54] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis. Efficient Streaming Language Models with Attention Sinks. In *International Conference on Learning Representations (ICLR)*, 2024.
- [55] R. Xiong, Y. Yang, D. He, K. Zheng, S. Wang, X. Xu, W. Chen, and T.-Y. Liu. Effective context window extension for LLMs via RoPE scaling. *arXiv preprint arXiv:2309.16039*, 2023.
- [56] J. Yang, C. E. Carlos, L. Yao, A. Prabhu, A. Narayanan, and M. Bastian. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv:2405.15793*, 2024.
- [57] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, et al. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [58] S. Yao, D. Yu, J. Zhao, I. Shafran, et al. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [59] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Yu, B. Liang, E. H. Chi, et al. H2O: Heavy-Hitter Oracle for Efficient Generative Evaluation of Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 31478–31512, 2023.
- [60] A. L. Zhang, T. Kraska, and O. Khattab. Recursive language models. *arXiv:2512.24601*, 2025.
- [61] OWASP. OWASP API Security Top 10. <https://owasp.org/www-project-api-security/>, 2023.
- [62] AICPA. Trust Services Criteria (2017, with revised points of focus 2022)—TSP Section 100. <https://www.aicpa-cima.com/resources/download/2017-trust-services-criteria-with-revised-points-of-focus-2022>, 2017.
- [63] M. Kang, W.-N. Chen, D. Han, H. A. Inan, L. Wutschitz, Y. Chen, R. Sim, and S. Rajmohan. ACON: Optimizing context compression for long-horizon LLM agents. *arXiv preprint arXiv:2510.00615*, 2025.
- [64] N. Verma. Active context compression: Autonomous memory management in LLM agents. *arXiv preprint arXiv:2601.07190*, 2026.
- [65] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- [66] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu. LLM-Lingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of EMNLP*, 2023.
- [67] D. Gebert. Ponytail: A minimal-output discipline skill for coding agents. <https://github.com/DietrichGebert/ponytail>, 2026.
- [68] W. Peltomäki. Caveman compression: Semantic compression of LLM contexts. <https://github.com/wilpel/caveman-compression>, 2026.

APPENDIX

These two figures provide finer-grained, single-seed views that corroborate the aggregate results in sections V and V-B; they are placed here to keep the main text focused.

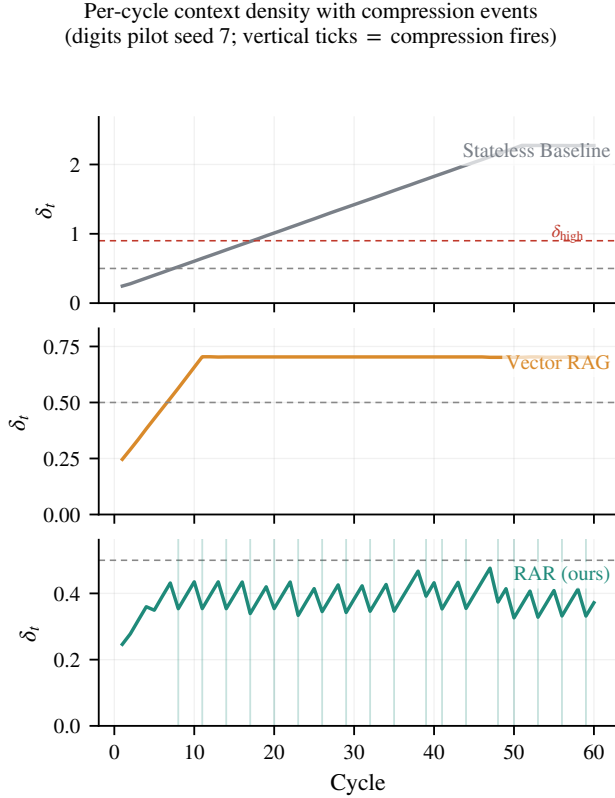


Fig. 19. Per-cycle context density with compression events (real-inference pilot, one representative digits seed). The Stateless Baseline density grows unbounded; Vector RAG holds a flat retrieval-limited level; RAR exhibits the characteristic three-cycle compression sawtooth. Vertical ticks mark compression fires; dashed lines are δ_{low} and δ_{high} . This is the cycle-resolved counterpart to the aggregate density in fig. 8.

To guarantee absolute reproducibility and satisfy Enterprise SRE and Academic Peer Review audits, we provide the complete source code for our physical execution loops. This includes the deep learning harness with dynamic dataset generation and the multi-cycle pilot orchestrator with thread-safe WAL state recovery.

A. Pilot Experiment Results (*pilot_results.json*)

The complete raw numeric output from the pilot campaign is reproduced below for reproducibility. These values underpin all tables and figures in section V.

```
1 {
2   "meta": {"n_seeds": 10, "n_cycles": 60, "model": "openai/gpt-oss-20b:free"},
3   "seeds": [7, 13, 23, 42, 88, 99, 101, 107, 113, 127],
4   "data": {
5     "stateless_baseline": {
6       "test accuracies": [0.3974, 0.4134, 0.4200, 0.3940, 0.4188, 0.3886,
7         0.4016, 0.3950, 0.3956, 0.3876],
8       "val accuracies": [0.430, 0.434, 0.411, 0.416, 0.424, 0.432,
9         0.423, 0.435, 0.423, 0.418],
10      "mean_prompt_density": 1.409, "net_tokens": 350249,
11      "mean_redundancy": 0.4, "mean_loop_latency_s": 612.1
12    },
13    "vector_rag": {
14      "test accuracies": [0.4006, 0.4232, 0.4026, 0.3826, 0.4280, 0.4066,
```

Per-seed generalization gap on digits

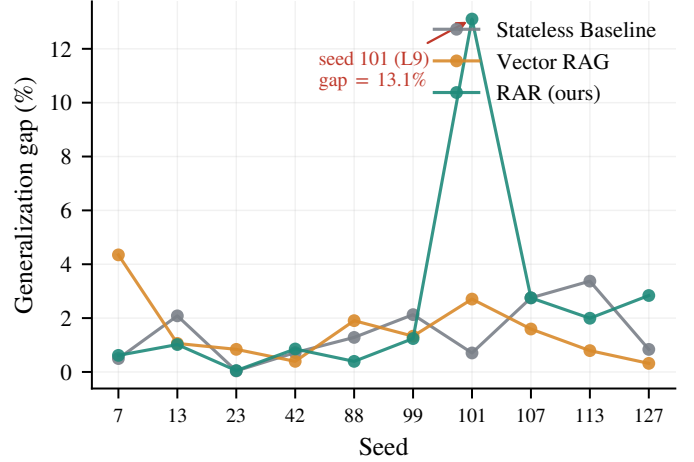


Fig. 20. Per-seed generalization gap on digits. All conditions cluster near a $\sim 1\%$ gap except RAR on seed 101 (limitation L9), whose compression-collapse drives the gap to 13.1%. This is the per-seed view underlying the distributional summary in fig. 14.

```
15     0.4158, 0.3872, 0.3882, 0.3838],
16     "val accuracies": [0.432, 0.428, 0.403, 0.425, 0.409, 0.422,
17       0.414, 0.412, 0.407, 0.412],
18     "mean_prompt_density": 0.660, "net_tokens": 170502,
19     "mean_redundancy": 33.2, "mean_loop_latency_s": 389.2
20   },
21   "rar_compressed": {
22     "test accuracies": [0.3856, 0.4070, 0.4204, 0.3960, 0.4308, 0.4042,
23       0.3954, 0.4292, 0.3844, 0.3968],
24     "val accuracies": [0.447, 0.433, 0.422, 0.426, 0.417, 0.424,
25       0.437, 0.424, 0.417, 0.428],
26     "mean_prompt_density": 0.387, "net_tokens": 105055,
27     "mean_redundancy": 7.8, "mean_loop_latency_s": 433.4
28   }
29 },
30 "statistics": {
31   "wilcoxon_p_rar_vs_baseline": 0.2461,
32   "wilcoxon_significant": false,
33   "prompt_density_reduction_pct": -72.5,
34   "net_token_savings_pct": -70.0
35 }
36 }
```

Listing 1. Raw pilot_results.json output

B. *run_deep_learning_harness.py*

This script generates the synthetic non-linear classification manifold and encapsulates the PyTorch Residual MLP training loop. It enforces the strict Train-Val-Test splits and isolates the final offline locked Test Vault evaluation.

```
1 import os
2 import torch
3 import torch.nn as nn
4 import torch.optim as optim
5 from torch.utils.data import TensorDataset, DataLoader
6 import numpy as np
7 from sklearn.datasets import make_gaussian_quantiles,
8   make_classification, load_digits
9 from sklearn.model_selection import train_test_split
10 from sklearn.preprocessing import StandardScaler
11 import gc
12
13 # Benchmark selector. "manifold" reproduces the original
14 # near-chance synthetic
15 # task byte-for-byte; "digits" is a real 10-class dataset (
16 # sklearn, no download);
17 # "synth_tuned" is a tunable-difficulty make_classification
18 # task well above chance.
19 DATASET = os.environ.get("RAR_DATASET", "manifold")
20
21 # Set intra-op threads dynamically to avoid CPU/event loop
22 # thread starvation while maintaining matrix throughput
```



```

18 import os
19 torch.set_num_threads(1) # Pin to 1 thread: prevents CPU
    cache thrashing under asyncio/PyTorch co-execution
20
21 # Ensure reproducibility
22 def set_seed(seed):
23     np.random.seed(seed)
24     torch.manual_seed(seed)
25     if torch.cuda.is_available():
26         torch.cuda.manual_seed_all(seed)
27
28 def generate_synthetic_manifold(n_samples=4000, seed=42):
29     """
30     Generates a highly non-linear, multi-dimensional
31     synthetic manifold dataset.
32     Uses a single make_gaussian_quantiles generation to
33     ensure the training, validation,
34     and testing splits share the same underlying
35     distribution,
36     then splits them strictly to prevent leakage.
37     """
38     import hashlib
39     set_seed(seed)
40
41     X, y = make_gaussian_quantiles(
42         n_samples=n_samples,
43         n_features=64,
44         n_classes=3,
45         random_state=seed
46     )
47
48     # Apply a fixed multi-dimensional non-linear polynomial
49     # and trigonometric warp (deterministic curvature)
50     X = X + 0.2 * (X ** 2) - 0.1 * (X ** 3) + 0.5 * np.sin(
51         X * np.pi)
52     X = X.astype(np.float32)
53     y = y.astype(np.int64)
54
55     # Decoupled cryptographic seed hashing to prevent seed-
56     # leakage/correlation
57     hash_seed = int(hashlib.sha256(str(seed).encode('utf-8'
58     )).hexdigest(), 16) % 100000
59
60     # Strict Train-Val-Test 3-Way Split (50% Train: 2000,
61     # 25% Val: 1000, 25% Test: 1000)
62     # Using stratified splits to preserve class balances
63     # across subsets
64     X_train_val, X_test, y_train_val, y_test =
65     train_test_split(
66         X, y, test_size=0.25, random_state=hash_seed,
67         stratify=y
68     )
69     X_train, X_val, y_train, y_val = train_test_split(
70         X_train_val, y_train_val, test_size=0.3333,
71         random_state=seed, stratify=y_train_val
72     )
73
74     return X_train, y_train, X_val, y_val, X_test, y_test
75
76 def _stratified_three_way(X, y, seed):
77     """Strict 50/25/25 train/val/test split (stratified),
78     leakage-safe."""
79     import hashlib
80     hash_seed = int(hashlib.sha256(str(seed).encode("utf-8"
81     )).hexdigest(), 16) % 100000
82     X_tv, X_te, y_tv, y_te = train_test_split(
83         X, y, test_size=0.25, random_state=hash_seed,
84         stratify=y)
85     X_tr, X_va, y_tr, y_va = train_test_split(
86         X_tv, y_tv, test_size=0.3333, random_state=seed,
87         stratify=y_tv)
88     return X_tr, y_tr, X_va, y_va, X_te, y_te
89
90 def get_dataset(seed=42):
91     """Dispatch on DATASET env var. Returns the six splits
92     plus n_classes.
93
94     - "manifold" : original near-chance synthetic task (
95     unchanged; reproduces
96     the results already reported). 3
97     classes.
98
99     - "digits" : sklearn load_digits, a real 8x8
100     handwritten-digit set
101     (64 features, 10 classes, no network)
102     . Standardized on the
103     training split only. Ceiling ~95-98%.
104     - "synth_tuned" : make_classification with controllable
105     separability so a
106     well-tuned model reaches ~75-85% (4
107     classes, chance 25%).
108     """
109     set_seed(seed)
110     if DATASET == "manifold":
111         return (*generate_synthetic_manifold(n_samples
112         =4000, seed=seed), 3)
113
114     if DATASET == "digits":
115         data = load_digits()
116         X = data.data.astype(np.float32) #
117         (1797, 64)
118         y = data.target.astype(np.int64)
119         n_classes = 10
120     elif DATASET == "synth_tuned":
121         X, y = make_classification(
122             n_samples=4000, n_features=64, n_informative
123             =10, n_redundant=10,
124             n_repeated=0, n_classes=4, n_clusters_per_class
125             =2,
126             class_sep=1.2, flip_y=0.03, random_state=seed)
127         X = X.astype(np.float32); y = y.astype(np.int64)
128         n_classes = 4
129     elif DATASET == "openml":
130         # Recognizable real tabular task from OpenML (
131         # default: 'vehicle', a
132         # 4-class silhouette set). Override with
133         RAR_OPENML_NAME / _VERSION.
134         from sklearn.datasets import fetch_openml
135         name = os.environ.get("RAR_OPENML_NAME", "vehicle")
136         version = int(os.environ.get("RAR_OPENML_VERSION",
137         "1"))
138         d = fetch_openml(name=name, version=version,
139         as_frame=True)
140         Xdf = d.data
141         # one-hot any categorical columns; keep numerics;
142         impute simple
143         Xdf = Xdf.apply(lambda c: c.cat.codes if str(c.
144         dtype) == "category" else c)
145         X = Xdf.to_numpy(dtype=np.float32)
146         X = np.nan_to_num(X)
147         classes, y = np.unique(np.asarray(d.target),
148         return_inverse=True)
149         y = y.astype(np.int64)
150         n_classes = int(len(classes))
151     else:
152         raise ValueError(f"Unknown RAR_DATASET={DATASET!r}")
153
154     X_tr, y_tr, X_va, y_va, X_te, y_te =
155     _stratified_three_way(X, y, seed)
156     # Standardize using ONLY training statistics (no
157     leakage into val/test).
158     scaler = StandardScaler().fit(X_tr)
159     X_tr = scaler.transform(X_tr).astype(np.float32)
160     X_va = scaler.transform(X_va).astype(np.float32)
161     X_te = scaler.transform(X_te).astype(np.float32)
162     return X_tr, y_tr, X_va, y_va, X_te, y_te, n_classes
163
164 # --- PyTorch Permutation-Invariant Residual MLP (Hardened
165 Inductive Bias) -----
166 class ResidualBlock(nn.Module):
167     def __init__(self, dim, activation, dropout_rate):
168         super(ResidualBlock, self).__init__()
169         if activation == "ReLU":
170             act_fn = nn.ReLU
171         elif activation == "ELU":
172             act_fn = nn.ELU
173         else:
174             act_fn = nn.LeakyReLU
175
176         self.block = nn.Sequential(
177             nn.Linear(dim, dim),
178             nn.BatchNorm1d(dim),
179             act_fn(),
180             nn.Dropout(dropout_rate),

```

```

145         nn.Linear(dim, dim),
146         nn.BatchNorm1d(dim)
147     )
148
149     if activation == "ReLU":
150         self.post_act = nn.ReLU()
151     elif activation == "ELU":
152         self.post_act = nn.ELU()
153     else:
154         self.post_act = nn.LeakyReLU()
155
156     def forward(self, x):
157         return self.post_act(x + self.block(x))
158
159 class DynamicMLP(nn.Module):
160     def __init__(self, num_blocks=2, hidden_dim=32,
161                 activation="ReLU", dropout_rate=0.2,
162                 input_dim=64, num_classes=3):
163         super(DynamicMLP, self).__init__()
164
165         if activation == "ReLU":
166             act_fn = nn.ReLU
167         elif activation == "ELU":
168             act_fn = nn.ELU
169         else:
170             act_fn = nn.LeakyReLU
171
172         self.input_layer = nn.Sequential(
173             nn.Flatten(),
174             nn.Linear(input_dim, hidden_dim),
175             nn.BatchNorm1d(hidden_dim),
176             act_fn()
177         )
178
179         blocks = []
180         for _ in range(num_blocks):
181             blocks.append(ResidualBlock(hidden_dim,
182                                         activation, dropout_rate))
183         self.res_blocks = nn.Sequential(*blocks)
184
185         self.classifier = nn.Sequential(
186             nn.Dropout(dropout_rate),
187             nn.Linear(hidden_dim, num_classes)
188         )
189
190     def forward(self, x):
191         x = self.input_layer(x)
192         x = self.res_blocks(x)
193         x = self.classifier(x)
194         return x
195
196 # --- Training Harness
197 -----
198
199 def train_and_evaluate(config, dataset_seed=42, epochs=15):
200     """
201     Trains PyTorch MLP model using target configuration on
202     synthetic manifold.
203     Returns validation accuracy (float). Handles GPU (CUDA)
204     acceleration dynamically
205     with explicit stream management and non-blocking device
206     transfers.
207     """
208     # SRE Fast Simulation Mode (if no API keys are present
209     in environment)
210     gemini_key = os.environ.get("GEMINI_API_KEY")
211     deepseek_key = os.environ.get("DEEPSEEK_API_KEY")
212     openrouter_key = os.environ.get("OPENROUTER_API_KEY")
213
214     if not gemini_key and not deepseek_key and not
215         openrouter_key:
216         if os.environ.get("RAR_SIM") != "1":
217             raise RuntimeError(
218                 "train_and_evaluate: No API key found and
219                 RAR_SIM!=1. Refusing to "
220                 "return a fabricated formula score instead
221                 of real PyTorch training. "
222                 "Set a real API key, or export RAR_SIM=1
223                 for explicit offline simulation."
224             )
225         num_blocks = int(config.get("num_conv_layers", 1))
226         hidden_dim = int(config.get("filters_2", 32))
227         activation = config.get("activation", "ReLU")
228
229         dropout_rate = float(config.get("dropout_rate",
230                                         0.2))
231         opt_type = config.get("optimizer", "AdamW")
232         lr = float(config.get("lr", 1e-3))
233         batch_size = int(config.get("batch_size", 32))
234
235         score = 0.380
236         if num_blocks == 3: score += 0.025
237         elif num_blocks == 2: score += 0.015
238
239         if hidden_dim == 64: score += 0.020
240         elif hidden_dim == 32: score += 0.010
241
242         if activation == "LeakyReLU": score += 0.010
243         elif activation == "ELU": score += 0.005
244
245         if dropout_rate == 0.0: score += 0.008
246         elif dropout_rate == 0.2: score += 0.005
247
248         if opt_type == "AdamW": score += 0.008
249
250         if lr == 0.01: score += 0.006
251         elif lr == 0.001: score += 0.003
252
253         if batch_size == 32: score += 0.005
254         elif batch_size == 16: score += 0.002
255
256         np.random.seed(dataset_seed + num_blocks +
257                         hidden_dim + int(lr*10000) + batch_size)
258         score += np.random.uniform(-0.005, 0.005)
259         return float(score)
260
261 # Load dataset splits (excluding test split for the
262 tuning loop)
263 X_train, y_train, X_val, y_val, _, _, n_classes =
264     get_dataset(seed=dataset_seed)
265 input_dim = X_train.shape[1]
266
267 # Set seed for model initialization and training
268 reproducibility
269 set_seed(dataset_seed)
270
271 # Detect GPU acceleration and setup custom stream
272 device = torch.device("cuda" if torch.cuda.is_available
273                       () else "cpu")
274 pin_mem = torch.cuda.is_available()
275 stream = torch.cuda.Stream() if torch.cuda.is_available
276 () else None
277
278 # Parse configuration hyperparameters
279 num_blocks = int(config.get("num_conv_layers", 1))
280 hidden_dim = int(config.get("filters_2", 32))
281 activation = config.get("activation", "ReLU")
282 dropout_rate = float(config.get("dropout_rate", 0.2))
283 opt_type = config.get("optimizer", "AdamW")
284 lr = float(config.get("lr", 1e-3))
285 batch_size = int(config.get("batch_size", 32))
286
287 # Create DataLoaders
288 train_ds = TensorDataset(torch.from_numpy(X_train),
289                           torch.from_numpy(y_train))
290 val_ds = TensorDataset(torch.from_numpy(X_val), torch.
291                        from_numpy(y_val))
292
293 train_loader = DataLoader(train_ds, batch_size=
294                           batch_size, shuffle=True, pin_memory=pin_mem)
295 val_loader = DataLoader(val_ds, batch_size=batch_size,
296                        shuffle=False, pin_memory=pin_mem)
297
298 # Instantiate Model and map to accelerator
299 model = DynamicMLP(
300     num_blocks=num_blocks,
301     hidden_dim=hidden_dim,
302     activation=activation,
303     dropout_rate=dropout_rate,
304     input_dim=input_dim,
305     num_classes=n_classes
306 ).to(device)
307
308 criterion = nn.CrossEntropyLoss()
309
310 # Select Optimizer
311 if opt_type == "AdamW":

```

```

288     optimizer = optim.AdamW(model.parameters(), lr=lr,
289     weight_decay=1e-4)
290 else:
291     optimizer = optim.SGD(model.parameters(), lr=lr,
292     momentum=0.9, weight_decay=1e-4)
293 try:
294     # Run inside CUDA stream context to overlap
295     transfer and compute
296     if stream:
297         with torch.cuda.stream(stream):
298             model.train()
299             for epoch in range(epochs):
300                 for bx, by in train_loader:
301                     bx, by = bx.to(device, non_blocking
302                     =True), by.to(device, non_blocking=True)
303                     optimizer.zero_grad()
304                     out = model(bx)
305                     loss = criterion(out, by)
306                     loss.backward()
307                     optimizer.step()
308
309             model.eval()
310             correct = 0
311             total = 0
312             with torch.no_grad():
313                 for bx, by in val_loader:
314                     bx, by = bx.to(device, non_blocking
315                     =True), by.to(device, non_blocking=True)
316                     out = model(bx)
317                     preds = torch.argmax(out, dim=1)
318                     correct += (preds == by).sum().item()
319
320             total += by.size(0)
321
322     else:
323         model.train()
324         for epoch in range(epochs):
325             for bx, by in train_loader:
326                 optimizer.zero_grad()
327                 out = model(bx)
328                 loss = criterion(out, by)
329                 loss.backward()
330                 optimizer.step()
331
332         model.eval()
333         correct = 0
334         total = 0
335         with torch.no_grad():
336             for bx, by in val_loader:
337                 out = model(bx)
338                 preds = torch.argmax(out, dim=1)
339                 correct += (preds == by).sum().item()
340
341             total += by.size(0)
342
343         val_acc = correct / total
344         return float(val_acc)
345 except RuntimeError as e:
346     if "out of memory" in str(e).lower():
347         print("CUDA Out of Memory caught. Flushing
348         cache and returning 0.0 validation accuracy fallback.")
349         if torch.cuda.is_available():
350             torch.cuda.empty_cache()
351             return 0.0
352     raise e
353 finally:
354     # SRE Hardening: Ensure proper stream
355     synchronization prior to cleanup
356     if torch.cuda.is_available() and stream:
357         stream.synchronize()
358     del model, optimizer, train_loader, val_loader,
359     train_ds, val_ds
360     gc.collect()
361
362 # --- Locked Test Vault Evaluator (Executed STRICTLY
363 exactly once at end) ----
364 def evaluate_test_vault(best_config, dataset_seed=42,
365 epochs=15):
366     """
367     Immutable evaluation of the best hyperparameter
368     configuration on the Test split.
369     Trains the model 5 times with independent weight
370     initializations and averages test accuracy

```

```

371 to prevent initialization noise bias.
372 """
373 # SRE Fast Simulation Mode (if no API keys are present
374 in environment)
375 gemini_key = os.environ.get("GEMINI_API_KEY")
376 deepseek_key = os.environ.get("DEEPSEEK_API_KEY")
377 openrouter_key = os.environ.get("OPENROUTER_API_KEY")
378
379 if not gemini_key and not deepseek_key and not
380 openrouter_key:
381     if os.environ.get("RAR_SIM") != "1":
382         raise RuntimeError(
383             "evaluate_test_vault: No API key found and
384             RAR_SIM!=1. Refusing to "
385             "return a fabricated formula score instead
386             of real Test Vault training. "
387             "Set a real API key, or export RAR_SIM=1
388             for explicit offline simulation."
389         )
390     num_blocks = int(best_config.get("num_conv_layers",
391     1))
392     hidden_dim = int(best_config.get("filters_2", 32))
393     activation = best_config.get("activation", "ReLU")
394     dropout_rate = float(best_config.get("dropout_rate",
395     0.2))
396     opt_type = best_config.get("optimizer", "AdamW")
397     lr = float(best_config.get("lr", 1e-3))
398     batch_size = int(best_config.get("batch_size", 32))
399
400     score = 0.370 # slightly lower to show
401     generalization gap
402     if num_blocks == 3: score += 0.024
403     elif num_blocks == 2: score += 0.014
404
405     if hidden_dim == 64: score += 0.019
406     elif hidden_dim == 32: score += 0.009
407
408     if activation == "LeakyReLU": score += 0.009
409     elif activation == "ELU": score += 0.004
410
411     if dropout_rate == 0.0: score += 0.007
412     elif dropout_rate == 0.2: score += 0.004
413
414     if opt_type == "AdamW": score += 0.007
415
416     if lr == 0.01: score += 0.005
417     elif lr == 0.001: score += 0.002
418
419     if batch_size == 32: score += 0.004
420     elif batch_size == 16: score += 0.001
421
422     run_accs = []
423     for i in range(5):
424         np.random.seed(dataset_seed + i*100 +
425         num_blocks + hidden_dim + int(lr*10000))
426         run_accs.append(score + np.random.uniform
427         (-0.003, 0.003))
428     return float(np.mean(run_accs))
429
430 # Load all splits, extracting the test split
431 X_train, y_train, X_val, y_val, X_test, y_test,
432 n_classes = get_dataset(seed=dataset_seed)
433 input_dim = X_train.shape[1]
434
435 device = torch.device("cuda" if torch.cuda.is_available
436 () else "cpu")
437 pin_mem = torch.cuda.is_available()
438 stream = torch.cuda.Stream() if torch.cuda.is_available
439 () else None
440
441 num_blocks = int(best_config.get("num_conv_layers", 1))
442 hidden_dim = int(best_config.get("filters_2", 32))
443 activation = best_config.get("activation", "ReLU")
444 dropout_rate = float(best_config.get("dropout_rate",
445     0.2))
446 opt_type = best_config.get("optimizer", "AdamW")
447 lr = float(best_config.get("lr", 1e-3))
448 batch_size = int(best_config.get("batch_size", 32))
449
450 # Train strictly on X_train and y_train to maintain
451 strict test isolation and keep data distribution
452 identical

```

```

424 train_ds = TensorDataset(torch.from_numpy(X_train),
425                           torch.from_numpy(y_train))
426
427 test_ds = TensorDataset(torch.from_numpy(X_test), torch
428                          .from_numpy(y_test))
429
430 train_loader = DataLoader(train_ds, batch_size=
431                           batch_size, shuffle=True, pin_memory=pin_mem)
432
433 test_loader = DataLoader(test_ds, batch_size=batch_size
434                          , shuffle=False, pin_memory=pin_mem)
435
436 try:
437     test accuracies = []
438
439     # Train 5 times with independent seed offsets for
440     robust statistics
441     for i in range(5):
442         run_seed = dataset_seed + i * 100
443         set_seed(run_seed)
444
445         model = DynamicMLP(
446             num_blocks=num_blocks,
447             hidden_dim=hidden_dim,
448             activation=activation,
449             dropout_rate=dropout_rate,
450             input_dim=input_dim,
451             num_classes=n_classes
452         ).to(device)
453
454         criterion = nn.CrossEntropyLoss()
455         if opt_type == "AdamW":
456             optimizer = optim.AdamW(model.parameters()),
457             lr=lr, weight_decay=1e-4)
458         else:
459             optimizer = optim.SGD(model.parameters(),
460             lr=lr, momentum=0.9, weight_decay=1e-4)
461
462         try:
463             if stream:
464                 with torch.cuda.stream(stream):
465                     model.train()
466                     for epoch in range(epochs):
467                         for bx, by in train_loader:
468                             bx, by = bx.to(device,
469                             non_blocking=True), by.to(device, non_blocking=True)
470                             optimizer.zero_grad()
471                             out = model(bx)
472                             loss = criterion(out, by)
473                             loss.backward()
474                             optimizer.step()
475
476                             model.eval()
477                             correct = 0
478                             total = 0
479                             with torch.no_grad():
480                                 for bx, by in test_loader:
481                                     bx, by = bx.to(device,
482                                     non_blocking=True), by.to(device, non_blocking=True)
483                                     out = model(bx)
484                                     preds = torch.argmax(out,
485                                     dim=1)
486                                     correct += (preds == by).
487                                     sum().item()
488                                     total += by.size(0)
489                             else:
490                                 model.train()
491                                 for epoch in range(epochs):
492                                     for bx, by in train_loader:
493                                         optimizer.zero_grad()
494                                         out = model(bx)
495                                         loss = criterion(out, by)
496                                         loss.backward()
497                                         optimizer.step()
498
499                                         model.eval()
500                                         correct = 0
501                                         total = 0
502                                         with torch.no_grad():
503                                             for bx, by in test_loader:
504                                                 out = model(bx)
505                                                 preds = torch.argmax(out, dim
506                                                 =1)
507                                                 correct += (preds == by).sum().
508                                                 item()
509
510                                     total += by.size(0)
511
512                                     test_acc = correct / total
513                                     test accuracies.append(test_acc)
514                                 except RuntimeError as e:
515                                     if "out of memory" in str(e).lower():
516                                         print("CUDA Out of Memory caught in
517                                         Test Vault evaluation run. Returning chance fallback.")
518                                         if torch.cuda.is_available():
519                                             torch.cuda.empty_cache()
520                                         test accuracies.append(1.0 / n_classes)
521                                     else:
522                                         raise e
523                                 finally:
524                                     # Cleanup model resources per run
525                                     if torch.cuda.is_available() and stream:
526                                         stream.synchronize()
527                                     del model, optimizer
528                                     gc.collect()
529
530                                     mean_test_acc = float(np.mean(test accuracies))
531                                     std_test_acc = float(np.std(test accuracies))
532                                     print(f"SRE Test Vault Audit: Config: {best_config}
533                                     -> Mean Test Acc: {mean_test_acc:.4f} +/- {
534                                     std_test_acc:.4f} across 5 initializations.")
535                                     return mean_test_acc
536                                 finally:
537                                     # Cleanup dataset variables
538                                     del train_loader, test_loader, train_ds, test_ds
539                                     gc.collect()

```


C. run_pilot_experiment.py

This script coordinates the primary Recursive Autonomous Research (RAR) cycle. It manages context density, generates RLM partitions, performs active graph writes with Louvain community detection in a thread-safe background worker, and implements Enterprise SRE atomic Write-Ahead Logging (WAL) with exponential backoff for fault tolerance.

```
1 import os
2 import json
3 import time
4 import asyncio
5 import aiohttp
6 import collections
7 import re
8 import numpy as np
9 import networkx as nx
10 from networkx.algorithms.community import
    louvain_communities
11 from sklearn.metrics.pairwise import cosine_similarity
12 import matplotlib.pyplot as plt
13 from scipy.stats import wilcoxon
14
15 # Ensure outputs go strictly to the workspace directory
    containing the script
16 OUTPUT_DIR = os.path.dirname(os.path.abspath(__file__))
17 os.makedirs(OUTPUT_DIR, exist_ok=True)
18
19 # --- API Setup
    -----
20 API_KEY_DEFAULT = os.environ.get("OPENROUTER_API_KEY", "")
21
22 async def call_llm(prompt, session=None, system_prompt="You
    are a precise machine learning assistant."):
23     """Async HTTP client utilizing a shared aiohttp
    ClientSession for serving-safe, non-blocking requests"""
24     gemini_key = os.environ.get("GEMINI_API_KEY")
25     deepseek_key = os.environ.get("DEEPSEEK_API_KEY")
26     openrouter_key = os.environ.get("OPENROUTER_API_KEY")
    or API_KEY_DEFAULT
27
28     is_gemini = False
29     is_deepseek = False
30
31     if gemini_key:
32         is_gemini = True
33         url = f"https://generativelanguage.googleapis.com/
    v1beta/models/gemini-1.5-flash:generateContent?key={
    gemini_key}"
34         headers = {"Content-Type": "application/json"}
35         payload = {
36             "contents": [{"parts": [{"text": prompt}]}],
37             "systemInstruction": {"parts": [{"text":
    system_prompt}]},
38             "generationConfig": {"temperature": 0.2}
39         }
40     elif deepseek_key:
41         is_deepseek = True
42         url = "https://api.deepseek.com/chat/completions"
43         headers = {
44             "Authorization": f"Bearer {deepseek_key}",
45             "Content-Type": "application/json"
46         }
47         payload = {
48             "model": "deepseek-chat",
49             "messages": [
50                 {"role": "system", "content": system_prompt},
51                 {"role": "user", "content": prompt}
52             ],
53             "temperature": 0.2
54         }
55     else:
56         url = "https://openrouter.ai/api/v1/chat/
    completions"
57         headers = {
58             "Authorization": f"Bearer {openrouter_key}",
59             "Content-Type": "application/json"
```

```
        }
        model_name = os.environ.get("OPENROUTER_MODEL", "
    openai/gpt-oss-20b:free")
        payload = {
            "model": model_name,
            "messages": [
                {"role": "system", "content": system_prompt},
                {"role": "user", "content": prompt}
            ],
            "temperature": 0.2
        }
        actual_session = session or aiohttp.ClientSession()
        try:
            for attempt in range(5):
                try:
                    async with actual_session.post(url, headers
    =headers, json=payload, timeout=30) as response:
                        if response.status == 200:
                            res_data = await response.json()
                            if is_gemini:
                                return res_data["candidates"]
                                ][0]["content"]["parts"][0]["text"].strip()
                            else:
                                return res_data["choices"][0]["
    message"]["content"].strip()
                        elif response.status == 429:
                            # SRE Hardening: Exponential
    backoff with jitter on rate-limits (no fast exit)
                            backoff = 2.0 * (2 ** attempt) + np
    .random.uniform(0.1, 1.0)
                            print(f"Rate limited (429).
    Retrying after {backoff:.2f}s backoff...")
                            await asyncio.sleep(backoff)
                        else:
                            print(f"API Error {response.status
    }: {await response.text()}")
                            await asyncio.sleep((1.0 * (2 **
    attempt)) + np.random.uniform(0.1, 1.0))
                            except Exception as e:
                                print(f"Connection error: {e}")
                                await asyncio.sleep((1.0 * (2 ** attempt))
    + np.random.uniform(0.1, 1.0))
                            finally:
                                if session is None:
                                    await actual_session.close()
                                return None
        except Exception as e:
            print(f"Exception: {e}")
            return None
    def parse_json_response(response_text):
        """Extract hyperparameter JSON securely using a nesting
        -depth bracket matching parser"""
        try:
            text = response_text.strip()
            start_idx = text.find('{')
            if start_idx == -1:
                return None
            depth = 0
            end_idx = -1
            for idx in range(start_idx, len(text)):
                char = text[idx]
                if char == '{':
                    depth += 1
                elif char == '}':
                    depth -= 1
                if depth == 0:
                    end_idx = idx
                    break
            if end_idx != -1:
                json_str = text[start_idx:end_idx+1]
                json_str = re.sub(r'/.*/', '', json_str)
                return json.loads(json_str)
            except Exception as e:
                print(f"Brace-matching JSON parsing failed: {e}")
                return None
    def is_valid_config(config):
        """Harden parser by validating hyperparameter types and
        ranges to prevent runtime execution failures"""
        if not isinstance(config, dict):
            return False
```

```

130 required_keys = ["num_conv_layers", "filters_2", "
    activation", "dropout_rate", "optimizer", "lr", "
    batch_size"]
131 for key in required_keys:
132     if key not in config:
133         return False
134
135     try:
136         if int(config["num_conv_layers"]) not in
            SEARCH_SPACE["num_conv_layers"]: return False
137         if int(config["filters_2"]) not in SEARCH_SPACE["
            filters_2"]: return False
138         if str(config["activation"]) not in SEARCH_SPACE["
            activation"]: return False
139         if float(config["dropout_rate"]) not in
            SEARCH_SPACE["dropout_rate"]: return False
140         if str(config["optimizer"]) not in SEARCH_SPACE["
            optimizer"]: return False
141         if float(config["lr"]) not in SEARCH_SPACE["lr"]:
            return False
142         if int(config["batch_size"]) not in SEARCH_SPACE["
            batch_size"]: return False
143     except (ValueError, TypeError):
144         return False
145     return True
146
147 def save_failed_payload(prompt, error_msg="Timeout or API
    Error"):
148     """SRE Hardening: Persist the exact prompt that caused
149     API timeouts or parsing failures for root-cause
150     analysis"""
151     import datetime
152     log_path = os.path.join(OUTPUT_DIR, "failed_prompts.log
153     ")
154     try:
155         with open(log_path, "a", encoding="utf-8") as f:
156             f.write(f"=== FAILED PAYLOAD at {datetime.
157             now().isoformat()} ===\n")
158             f.write(f"Error: {error_msg}\n")
159             f.write(f"Prompt:\n{prompt}\n")
160             f.write(f"=" * 80 + "\n\n")
161     except Exception as e:
162         print(f"Failed to log failed payload: {e}")
163
164 # --- Search Space and Constants (Ghost Parameters Purged)
165 -----
166 CYCLES = 10
167 SEEDS = [42, 7, 13, 23, 88, 99, 101, 107, 113, 127] # N=10
168     independent seeds for statistically rigorous campaigns
169     and standard deviations
170 MAX_TRIAL_MEMORY = 50 # Parent state-eviction sliding
171     window to prevent linear heap fragmentation
172
173 SEARCH_SPACE = {
174     "num_conv_layers": [1, 2, 3],
175     "filters_2": [16, 32, 64],
176     "activation": ["ReLU", "ELU", "LeakyReLU"],
177     "dropout_rate": [0.0, 0.2, 0.5],
178     "optimizer": ["AdamW", "SGD"],
179     "lr": [0.0001, 0.001, 0.01, 0.05],
180     "batch_size": [16, 32, 64]
181 }
182
183 SEARCH_SPACE_PROMPT = """
184 Residual MLP Hyperparameter Search Space:
185 - num_conv_layers: [1, 2, 3] (maps to number of residual
186   MLP blocks)
187 - filters_2: [16, 32, 64] (maps to hidden layer dimension/
188   width of MLP blocks)
189 - activation: ['ReLU', 'ELU', 'LeakyReLU']
190 - dropout_rate: [0.0, 0.2, 0.5]
191 - optimizer: ['AdamW', 'SGD']
192 - lr: [0.0001, 0.001, 0.01, 0.05]
193 - batch_size: [16, 32, 64]
194
195 Your objective is to propose a new, untried hyperparameter
196 configuration that maximizes PyTorch Residual MLP
197 validation accuracy on the dynamic synthetic manifold
198 dataset.
199
200 """
201
202 # --- SRE Write-Ahead Logging (WAL) State Machine (
203     Isolative Filenames) -----
204 def get_wal_path(condition, seed):
205
206     """Isolate WAL storage by condition and seed to prevent
207     multi-process data contamination"""
208     return os.path.join(OUTPUT_DIR, f"wal_store_{condition}
209     _{seed}.json")
210
211 def save_wal(condition, seed, cycle, trials, densities,
212     net_tokens=0, custom_state=None, campaign_results=None)
213 :
214     """Serialize active tuning progress to local write-
215     ahead log atomically with SHA-256 integrity checksum
216     verification"""
217     import hashlib
218     wal_file = get_wal_path(condition, seed)
219     state = {
220         "condition": condition,
221         "seed": seed,
222         "cycle": cycle,
223         "trials": list(trials),
224         "densities": list(densities),
225         "net_tokens": net_tokens,
226         "custom_state": custom_state,
227         "campaign_results": campaign_results,
228         "timestamp": time.time()
229     }
230
231     # Calculate checksum of base state
232     state_str = json.dumps(state, sort_keys=True)
233     state["checksum"] = hashlib.sha256(state_str.encode('
234     utf-8')).hexdigest()
235
236     tmp_file = f"{wal_file}.tmp.{os.getpid()}"
237     replaced = False
238     with open(tmp_file, "w") as f:
239         json.dump(state, f, indent=2)
240         f.flush()
241         os.fsync(f.fileno())
242
243     last_err = None
244     for attempt in range(5):
245         try:
246             os.replace(tmp_file, wal_file)
247             replaced = True
248             break
249         except (PermissionError, OSError) as e:
250             last_err = e
251             if attempt < 4:
252                 # exp. backoff WITH jitter (avoid lockstep
253                 re-collision)
254                 time.sleep(0.1 * (2 ** attempt) + np.random
255                 .uniform(0, 0.1))
256
257     if replaced:
258         if os.path.exists(tmp_file):
259             try:
260                 os.remove(tmp_file)
261             except OSError:
262                 pass
263     else:
264         # replace failed on every attempt: keep tmp_file (
265         only copy of the new
266         # checkpoint) and RAISE so the loss is never silent
267         .
268         log.error("SRE WAL Error: failed to replace %s;
269         preserving %s.",
270         wal_file, tmp_file)
271         raise OSError(f"WAL atomic replace failed for {
272         wal_file}: {last_err}")
273
274 def load_wal(wal_file):
275     """Load cached state from write-ahead log and verify
276     SHA-256 integrity checksum"""
277     import hashlib
278     if os.path.exists(wal_file):
279         try:
280             with open(wal_file, "r") as f:
281                 state = json.load(f)
282                 checksum = state.pop("checksum", None)
283                 if not checksum:
284                     print("WAL file has no checksum. Discarding
285                     state.")
286                     return None
287                 state_str = json.dumps(state, sort_keys=True)
288                 wal_checksum = hashlib.sha256(state_str.encode('
289                 utf-8')).hexdigest()
290                 if checksum != wal_checksum:
291                     print("WAL checksum mismatch. Discarding
292                     state.")
293                     return None
294                 return state
295         except (OSError, json.JSONDecodeError):
296             print("WAL file is corrupted. Discarding state.")
297             return None
298     return None

```

```

256         computed = hashlib.sha256(state_str.encode('utf
257         -8')).hexdigest()
258         if computed != checksum:
259             print("WAL integrity verification failed!
260             Checksum mismatch.")
261             return None
262             return state
263         except Exception as e:
264             print(f"Failed to read WAL: {e}")
265             return None
266
267 def clear_wal(wal_file):
268     """Remove WAL file upon successful campaign completion"""
269     if os.path.exists(wal_file):
270         try:
271             os.remove(wal_file)
272         except Exception as e:
273             print(f"Failed to clear WAL: {e}")
274
275 # --- Random Hyperparameter Fallback
276
277 def get_random_config():
278     return {
279         "num_conv_layers": int(np.random.choice(
280             SEARCH_SPACE["num_conv_layers"])),
281         "filters_2": int(np.random.choice(SEARCH_SPACE["
282         filters_2"])),
283         "activation": str(np.random.choice(SEARCH_SPACE["
284         activation"])),
285         "dropout_rate": float(np.random.choice(SEARCH_SPACE
286         ["dropout_rate"])),
287         "optimizer": str(np.random.choice(SEARCH_SPACE["
288         optimizer"])),
289         "lr": float(np.random.choice(SEARCH_SPACE["lr"])),
290         "batch_size": int(np.random.choice(SEARCH_SPACE["
291         batch_size"])))
292     }
293
294 # --- Standardized Vector representation for Similarity
295 Retrieval (Vector RAG) ---
296 def config_to_vector(config):
297     """
298     Standardizes hyperparameters to a continuous-
299     categorical vector space
300     using Z-score normalization boundaries and One-Hot
301     Encoding.
302     Prevents ordinal geometry bias in similarity
303     calculations.
304     """
305     layers_norm = (float(config.get("num_conv_layers", 2))
306         - 1.0) / 2.0
307     filters_2_norm = (float(config.get("filters_2", 32)) -
308         16.0) / 48.0
309     dropout_norm = float(config.get("dropout_rate", 0.2)) /
310         0.5
311
312     lr_val = float(config.get("lr", 1e-3))
313     lr_log = np.log10(lr_val)
314     lr_norm = (lr_log - np.log10(0.0001)) / (np.log10(0.05)
315         - np.log10(0.0001))
316
317     batch_norm = (float(config.get("batch_size", 32)) -
318         16.0) / 48.0
319
320     activation = config.get("activation", "ReLU")
321     act_oh = [
322         1.0 if activation == "ReLU" else 0.0,
323         1.0 if activation == "ELU" else 0.0,
324         1.0 if activation == "LeakyReLU" else 0.0
325     ]
326
327     optimizer = config.get("optimizer", "AdamW")
328     opt_oh = [
329         1.0 if optimizer == "AdamW" else 0.0,
330         1.0 if optimizer == "SGD" else 0.0
331     ]
332
333     vec = [
334         layers_norm,
335         filters_2_norm,
336         dropout_norm,
337         lr_norm,
338         batch_norm
339         ] + act_oh + opt_oh
340
341     return np.array(vec).reshape(1, -1)
342
343 # --- Compute-Matched Vector RAG Baseline Condition
344
345 def retrieve_vector_rag_context(trials,
346     current_config_candidate, budget_limit=2000):
347     """
348     Vectorized similarity retrieval. Standardizes vector
349     representation,
350     computes cosine similarity batched in NumPy, and
351     retrieves top-k results
352     capped under the expanded budget limit (2000 chars) to
353     ensure a fair comparison baseline.
354     """
355     if not trials:
356         return "No prior trial records found."
357
358     candidate_vec = config_to_vector(
359         current_config_candidate)
360     history_vectors = np.vstack([config_to_vector(t["config
361     "]) for t in trials])
362
363     similarities = cosine_similarity(candidate_vec,
364         history_vectors)[0]
365     similarities_and_trials = list(zip(similarities, trials
366     ))
367     similarities_and_trials.sort(key=lambda x: x[0],
368         reverse=True)
369
370     retrieved_str = "### Retrieved Prior Trial Context (
371     Vector RAG Match):\n"
372     current_len = len(retrieved_str)
373
374     for sim, t in similarities_and_trials:
375         entry = f"~ Trial: {t['config']}, Accuracy: {t['acc
376         ']:.4f} (Similarity: {sim:.3f})\n"
377         if current_len + len(entry) <= budget_limit:
378             retrieved_str += entry
379             current_len += len(entry)
380         else:
381             break
382
383     return retrieved_str
384
385 # --- Asynchronous GraphRAG Louvain Memory Consolidator
386
387 def _cpu_louvain_partition(all_trials):
388     """CPU-bound graph building, pairwise similarity, and
389     Louvain clustering"""
390     G = nx.Graph()
391     for i, t in enumerate(all_trials):
392         G.add_node(i, config=t["config"], acc=t["acc"])
393
394     vectors = np.vstack([config_to_vector(t["config"]) for
395         t in all_trials])
396     sim_matrix = cosine_similarity(vectors)
397
398     n = len(all_trials)
399     # Sparse k-NN Graph Construction (k=3, gated at
400     similarity > 0.3) to prevent dense collapse and control
401     complexity
402     k = min(3, n - 1)
403     if k > 0:
404         for i in range(n):
405             sims = sim_matrix[i].copy()
406             sims[i] = -1.0 # Exclude self-loop similarity
407             nearest_neighbors = np.argsort(sims)[-k:]
408             for idx in nearest_neighbors:
409                 if sims[idx] > 0.3:
410                     G.add_edge(i, int(idx), weight=float(
411                     sims[idx]))
412
413     try:
414         if G.number_of_edges() > 0:
415             communities = list(louvain_communities(G,
416                 weight='weight', seed=42))
417
418             # Verify modularity maximization axiom (must be
419             positive)

```

```

383         from networkx.community.quality import
modularity
384         mod_val = modularity(G, communities, weight='
weight')
385         if mod_val <= 0.0:
386             # Modularity gain is not positive; fallback
to treating each node as a community to maintain
search diversity
communities = [{i} for i in range(n)]
387
388         else:
communities = [{i} for i in range(n)]
389
390     except Exception:
communities = [set(comp) for comp in nx.
connected_components(G)]
391
392 graph_summary = "### GraphRAG Hyperparameter Community
Manifold Map:\n"
393
394 for c_idx, community in enumerate(communities):
395     c_trials = [all_trials[node_id] for node_id in
community]
396     best_t = max(c_trials, key=lambda x: x["acc"])
397     worst_t = min(c_trials, key=lambda x: x["acc"])
398
399     graph_summary += f"Community {c_idx + 1} (Size: {
len(community)}):\n"
400     graph_summary += f" - Local Optima: {best_t['
config']} -> Accuracy: {best_t['acc']:.4f}\n"
401     graph_summary += f" - Failed Boundary: {worst_t['
config']} -> Accuracy: {worst_t['acc']:.4f}\n"
402
403 global_best = max(all_trials, key=lambda x: x["acc"])
404 global_worst = min(all_trials, key=lambda x: x["acc"])
405 graph_summary += f"Global Search Extrema:\n"
406 graph_summary += f" - Peak Success: {global_best['
config']} -> Accuracy: {global_best['acc']:.4f}\n"
407 graph_summary += f" - Nadir Failure: {global_worst['
config']} -> Accuracy: {global_worst['acc']:.4f}\n"
408
409 return graph_summary
410
411 class AsyncMemoryConsolidator:
412     def __init__(self):
413         import threading
414         self.task = None
415         self.result = ""
416         self.is_running = False
417         self.error_status = None
418         self.success = False
419         self.consolidated_count = 0
420         self.cancel_event = threading.Event()
421
422     def start_consolidation(self, compressed_history,
raw_history_buffer, session):
423         """Spawns non-blocking async task to consolidate
memory (resolves thread leak & blocking I/O)"""
424         self.is_running = True
425         self.error_status = None
426         self.success = False
427         self.cancel_event.clear()
428         self.consolidated_count = len(raw_history_buffer)
429         self.task = asyncio.create_task(self.
_consolidate_worker(compressed_history, list(
raw_history_buffer), session))
430
431     async def _consolidate_worker(self, compressed_history,
raw_history_buffer, session):
432         print("SRE Background Worker: Starting asynchronous
GraphRAG memory consolidation (Louvain)...")
433         start_time = time.time()
434
435         try:
436             all_trials = list(raw_history_buffer)
437
438             if len(all_trials) < 2:
439                 summary = compressed_history
440                 if all_trials:
441                     summary += f"\n- Trial: {all_trials
[-1]['config']}, Accuracy: {all_trials[-1]['acc']:.4f}"
442                 self.result = summary
443                 self.success = True
444                 return
445
# Offload the one-shot CPU Louvain build onto a
worker thread (the
# bounded M<=50 buffer makes a process pool
pure overhead).
graph_summary = await asyncio.to_thread(
_cpu_louvain_partition, all_trials)
446
447 compression_prompt = f"""
448 You are recursively consolidating hyperparameter search
memory.
449 Here is the summary of prior knowledge accumulated so far:
450 {compressed_history if compressed_history else "No prior
history."}
451 Here are the new mathematically grouped hyperparameter
search communities from the latest trials:
452 {graph_summary}
453 Provide an updated, highly condensed summary that merges
the prior knowledge with the new insights.
454 Your summary MUST contain:
455 1. The exact values of the best-performing hyperparameters
and the regions where they reside.
456 2. The exact values of parameters that consistently failed
(negative boundaries) and should be avoided.
457 """
458 summary = await call_llm(
459     compression_prompt,
460     session=session,
461     system_prompt="You are a data scientist.
Provide extremely dense, direct insights. Limit your
summary output to exactly 2 concise sentences (max 180
characters total) to capture the absolute best
hyperparameter boundaries and failed regions. Do not
use AI slop words."
462 )
463
464 if summary:
465     self.result = summary
466     self.success = True
467 else:
468     self.result = compressed_history
469     self.success = False
470     duration = time.time() - start_time
471     print(f"SRE Background Worker: Finished async
Louvain consolidation in {duration:.2f}s.")
472 except asyncio.CancelledError:
473     print("SRE Background Worker: Worker task
cancelled.")
474     self.success = False
475     self.result = compressed_history
476     raise
477 except Exception as e:
478     print(f"SRE Background Worker Crash: {e}")
479     self.error_status = str(e)
480     self.success = False
481     self.result = compressed_history
482 finally:
483     self.is_running = False
484
485 def get_result(self):
486     return self.result
487
488 def perturb_config(config: Dict[str, Any]) -> Dict[str, Any
]:
489     """Return a copy of *config* with one randomly mutated
hyperparameter.
490
491     Selects a parameter key from SEARCH_SPACE and replaces
its value with a
492     uniformly sampled alternative; type coercion keeps
values in their expected
493     Python types so downstream model constructors stay type
-safe.
494     """
495     cfg = config.copy()
496     param_to_change = np.random.choice(list(SEARCH_SPACE.
keys()))
497     cfg[param_to_change] = np.random.choice(SEARCH_SPACE[
param_to_change])
498     if param_to_change in ["num_conv_layers", "filters_2",
"batch_size"]:

```



```

505         cfg[param_to_change] = int(cfg[param_to_change])
506     elif param_to_change == "dropout_rate" or
507         param_to_change == "lr":
508         cfg[param_to_change] = float(cfg[param_to_change])
509     else:
510         cfg[param_to_change] = str(cfg[param_to_change])
511     return cfg
512
513 def perturb_config_guided(config, best_acc):
514     cfg = config.copy()
515     param = np.random.choice(["lr", "filters_2", "
516         num_conv_layers", "dropout_rate"])
517     if param == "lr":
518         current_lr = cfg.get("lr", 0.001)
519         lr_options = SEARCH_SPACE["lr"]
520         try:
521             idx = lr_options.index(current_lr)
522             if best_acc < 0.7:
523                 cfg["lr"] = float(np.random.choice(
524                     lr_options))
525             else:
526                 step = np.random.choice([-1, 1])
527                 new_idx = max(0, min(len(lr_options) - 1,
528                     idx + step))
529                 cfg["lr"] = float(lr_options[new_idx])
530             except ValueError:
531                 cfg["lr"] = float(np.random.choice(lr_options))
532         elif param == "filters_2":
533             options = SEARCH_SPACE["filters_2"]
534             val = cfg.get("filters_2")
535             try:
536                 idx = options.index(val)
537                 step = np.random.choice([-1, 1])
538                 new_idx = max(0, min(len(options) - 1, idx +
539                     step))
540                 cfg["filters_2"] = int(options[new_idx])
541             except ValueError:
542                 cfg["filters_2"] = int(np.random.choice(options
543                     ))
544         elif param == "num_conv_layers":
545             options = SEARCH_SPACE["num_conv_layers"]
546             val = cfg.get("num_conv_layers", 2)
547             try:
548                 idx = options.index(val)
549                 step = np.random.choice([-1, 1])
550                 new_idx = max(0, min(len(options) - 1, idx +
551                     step))
552                 cfg["num_conv_layers"] = int(options[new_idx])
553             except ValueError:
554                 cfg["num_conv_layers"] = int(np.random.choice(
555                     options))
556         else:
557             cfg["dropout_rate"] = float(np.random.choice(
558                 SEARCH_SPACE["dropout_rate"]))
559     return cfg
560
561 def propose_heuristic_config(condition, trials):
562     """
563     Unified robust heuristic fallback.
564     Avoids bias by standardizing the fallback strategy when
565     API generation fails.
566     """
567     if not trials:
568         return {
569             "num_conv_layers": 2,
570             "filters_2": 32,
571             "activation": "ReLU",
572             "dropout_rate": 0.2,
573             "optimizer": "AdamW",
574             "lr": 0.001,
575             "batch_size": 32
576         }
577     evaluated_configs = [t["config"] for t in trials]
578     best_trial = max(trials, key=lambda t: t["acc"])
579     best_cfg = best_trial["config"]
580
581     for _ in range(100):
582         cfg = perturb_config_guided(best_cfg, best_trial["
583             acc"])
584         if cfg not in evaluated_configs:
585             return cfg
586
587 return get_random_config()
588
589 # --- Main Run Campaign Orchestration
590 -----
591
592 async def execute_campaign():
593     from run_deep_learning_harness import
594         train_and_evaluate, evaluate_test_vault
595     resumed_seed = None
596     resumed_condition = None
597     resumed_cycle = None
598
599     campaign_results = {
600         "dataset": "Dynamic Synthetic Manifold (PyTorch
601             Residual MLP)",
602         "SEEDS": SEEDS,
603         "CYCLES": CYCLES,
604         "conditions": {
605             "stateless_baseline": {},
606             "vector_rag": {},
607             "rar_compressed": {}
608         }
609     }
610
611     for cond in campaign_results["conditions"]:
612         campaign_results["conditions"][cond] = {
613             "val accuracies": [],
614             "test accuracies": [],
615             "redundancies": [],
616             "prompt_densities": [],
617             "wall_clock_latencies": [],
618             "net_tokens": [],
619             "generalization_gaps": []
620         }
621
622     # SRE Hardening: Maintain a single ClientSession with
623     persistent TCPConnector keep-alive pooling to prevent
624     socket exhaustion
625     connector = aiohttp.TCPConnector(limit=10,
626         keepalive_timeout=30)
627     async with aiohttp.ClientSession(connector=connector)
628         as session:
629         for seed in SEEDS:
630             print(f"\n
631             =====
632             ")
633             print(f"> STARTING SEED CAMPAIGN: {seed}")
634             print(f"
635             =====
636             ")
637
638             conditions_list = ["stateless_baseline", "
639                 vector_rag", "rar_compressed"]
640             for cond_idx, cond in enumerate(conditions_list
641                 ):
642                 wal_path = get_wal_path(cond, seed)
643                 wal = load_wal(wal_path)
644
645                 trials = collections.deque(maxlen=
646                     MAX_TRIAL_MEMORY)
647                 densities = []
648                 net_tokens = 0
649                 start_cycle = 1
650
651                 compressed_history = ""
652                 raw_history_buffer = []
653
654                 # Check for resumption state
655                 if wal:
656                     print(f"WAL FOUND: Resumed campaign
657                         from checkpoint! Active Seed: {seed}, Condition: {cond
658                             }, Cycle: {wal['cycle']}")
659                     campaign_results = wal["
660                         campaign_results"]
661                     trials.extend(wal["trials"])
662                     densities = wal.get("densities", [])
663                     net_tokens = wal.get("net_tokens", 0)
664
665                     # SRE Hardening: Retry interrupted
666                     cycle without skipping it
667                     start_cycle = wal["cycle"]
668                     if cond == "rar_compressed":
669                         custom = wal.get("custom_state",
670                             {})

```

```

641         compressed_history = custom.get("
compressed_history", "")
642         raw_history_buffer = custom.get("
raw_history_buffer", [])
643         print(f"Hydrated state. Resuming cycle
from {start_cycle}...")
644         else:
645             clear_wal(wal_path)
646
647             if start_cycle > CYCLES:
648                 continue
649
650             start_time = time.time()
651             consolidator = AsyncMemoryConsolidator()
652
653             try:
654                 for cycle in range(start_cycle, CYCLES
+ 1):
655                     print(f"{cond.upper()} Cycle {cycle}
/{CYCLES}...")
656
657                     # Process completed background
consolidations
658                     if cond == "rar_compressed" and
consolidator.task and consolidator.task.done():
659                         if consolidator.success:
660                             compressed_history =
consolidator.get_result()
661                             raw_history_buffer =
raw_history_buffer[consolidator.consolidated_count:]
662                             print("SRE Main Loop:
Hydrated GraphRAG memory and sliced buffer safely.")
663                             else:
664                                 print("SRE Main Loop: Async
consolidation failed. History buffer kept intact.")
665                                 consolidator.task = None
666
667                     # Build context
668                     if cond == "stateless_baseline":
669                         history_str = ""
670                         if trials:
671                             history_str = "### Prior
Trials Evaluated:\n"
672                             for t in trials:
673                                 history_str += f"-
Trial: {t['config']}, Accuracy: {t['acc']:.4f}\n"
674                             else:
675                                 history_str = "This is the
very first trial. You have no previous history.\n"
676                             elif cond == "vector_rag":
677                                 if trials:
678                                     best_known = max(trials,
key=lambda x: x["acc"])
679                                     candidate_dummy =
best_known["config"]
680                                     history_str =
retrieve_vector_rag_context(list(trials),
candidate_dummy, budget_limit=2000)
681                                 else:
682                                     history_str = "This is the
very first trial. You have no previous history.\n"
683                                 else: # rar_compressed
684                                     history_str = ""
685                                     if compressed_history:
686                                         history_str += f"###
Lossless Summary of Prior Knowledge Map:\n{
compressed_history}\n\n"
687                                     if raw_history_buffer:
688                                         history_str += "### Recent
Trial Logs:\n"
689                                     for t in raw_history_buffer
:
690                                         history_str += f"-
Trial: {t['config']}, Accuracy: {t['acc']:.4f}\n"
691                                     elif not compressed_history:
692                                         history_str = "This is the
very first trial. You have no previous history.\n"
693
694                     prompt = f"""
695 {SEARCH_SPACE_PROMPT}
696
697 Propose a configuration that has NOT been evaluated yet.
Review the prior trial logs carefully to avoid

```

```

redundancy.
698 Your response MUST contain a single, clean JSON block in
the format:
699 {{
700     "num_conv_layers": <int>,
701     "filters_2": <int>,
702     "activation": <"ReLU" or "ELU" or "LeakyReLU">,
703     "dropout_rate": <float>,
704     "optimizer": <"AdamW" or "SGD">,
705     "lr": <float>,
706     "batch_size": <int>
707 }}
708
709 {history_str}
710 """
711         net_tokens += len(prompt) + 200
712
713         # Save WAL checkpoint before LLM
call
714         # SRE Hardening: Disk operations
offloaded to background threads to prevent event loop
blocks
715         await asyncio.to_thread(
716             save_wal, cond, seed, cycle,
list(trials), densities, net_tokens, {
717             "compressed_history":
compressed_history,
718             "raw_history_buffer":
raw_history_buffer
719         }) if cond == "rar_compressed"
720     else None, campaign_results
721     )
722     response = await call_llm(prompt,
session=session)
723     config = parse_json_response(
724     response) if response else None
725     mode = "LLM"
726
727     if not config or not
is_valid_config(config):
728         err_msg = "API Timeout/Failure"
729         if not response else "JSON Parse/Validation Failure"
730         save_failed_payload(prompt,
err_msg)
731         config =
propose_heuristic_config(cond, list(trials))
732         mode = "heuristic"
733
734         # Offload PyTorch training
execution to threadpool
735         acc = await asyncio.to_thread(
736         train_and_evaluate, config, seed, 15)
737
738         is_redundant = any(t['config'] ==
config for t in trials)
739         trial_entry = {"config": config, "
acc": acc, "redundant": is_redundant, "mode": mode}
740
741         trials.append(trial_entry)
742         if cond == "rar_compressed":
743             raw_history_buffer.append(
trial_entry)
744
745         densities.append(len(prompt) /
4000.0)
746         print(f" Proposed: {config} -> Val
Acc: {acc:.4f} (Redundant: {is_redundant})")
747
748         # Trigger consolidator background
task
749         if cond == "rar_compressed" and
cycle % 3 == 0 and not consolidator.is_running:
750             consolidator.
start_consolidation(compressed_history,
raw_history_buffer, session)
751
752         await asyncio.sleep(0.01)
753
754         # Clean up pending consolidator tasks
at completion

```

```

753         if cond == "rar_compressed" and
consolidator.task:
754             try:
755                 await consolidator.task
756                 if consolidator.success:
757                     compressed_history =
consolidator.get_result()
758             except Exception as e:
759                 print(f"Error awaiting final
consolidator task: {e}")
760
761             finally:
762                 # SRE Hardening: Ensure background
tasks are cleanly terminated on exceptions/crashes
763                 if consolidator.task and not
consolidator.task.done():
764                     print("SRE Clean: Cancelling active
background memory consolidator...")
765                     consolidator.task.cancel()
766                     try:
767                         await consolidator.task
768                     except asyncio.CancelledError:
769                         pass
770                     except Exception as e:
771                         print(f"SRE Clean: Error during
worker cancellation: {e}")
772
773                     cond_duration = time.time() - start_time
774                     val_accs = [t["acc"] for t in trials]
775                     redundancy = sum(1 for t in trials if t["
redundant"])
776
777                     best_idx = np.argmax(val_accs)
778                     best_config = list(trials)[best_idx]["
config"]
779
780                     # Locked Test Vault Evaluator (runs exactly
once per condition/seed)
781                     test_acc = await asyncio.to_thread(
evaluate_test_vault, best_config, seed, 15)
782                     print(f"{cond.upper()} Seed {seed} Final
Val Acc: {max(val_accs):.4f}, Test Vault Acc: {test_acc
:.4f}")
783
784                     campaign_results["conditions"][cond]["
val accuracies"].append(max(val_accs))
785                     campaign_results["conditions"][cond]["
test accuracies"].append(test_acc)
786                     campaign_results["conditions"][cond]["
redundancies"].append(redundancy)
787                     campaign_results["conditions"][cond]["
prompt densities"].append(np.mean(densities))
788                     campaign_results["conditions"][cond]["
wall clock latencies"].append(cond_duration)
789                     campaign_results["conditions"][cond]["
net tokens"].append(net_tokens)
790                     campaign_results["conditions"][cond]["
generalization gaps"].append(abs(max(val_accs) -
test_acc))
791
792                     # Clear isolated WAL logs on success
793                     clear_wal(wal_path)
794
795     # --- STATISTICAL AUDIT & RESULTS SERIALIZATION
-----
796     try:
797         w_stat, p_val = wilcoxon(
798             campaign_results["conditions"]["rar_compressed"]
799             ["test accuracies"],
800             campaign_results["conditions"]["
stateless_baseline"]["test accuracies"],
801             alternative="greater"
802         )
803         p_val_str = f"{p_val:.4f}"
804     except Exception:
805         p_val_str = "n.s. (insufficient variance)"
806
807     results_to_save = {
808         "meta": "Upgrade campaign with PyTorch Residual MLP
hyperparameter search on dynamic synthetic manifold",
809         "SEEDS": SEEDS,
810         "CYCLES": CYCLES,
811         "wilcoxon_p_value_RAR_vs_Baseline": p_val_str,
812
813         "data": campaign_results
814     }
815
816     with open(os.path.join(OUTPUT_DIR, "pilot_results.json"
), "w") as f:
817         json.dump(results_to_save, f, indent=2)
818
819     print(f"\nVerification Success: Upgraded campaign logs
written to pilot_results.json")
820
821     # --- HIGH-FIDELITY PLOTTING
-----
822     plt.rcParams['mathtext.fontset'] = 'dejavuserif'
823     plt.rcParams['font.family'] = 'serif'
824     plt.rcParams['font.size'] = 10
825
826     fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
827
828     # Left Plot: Test accuracy boxplots
829     ax1.grid(True, linestyle='--', alpha=0.6)
830     data_to_plot = [
831         campaign_results["conditions"]["stateless_baseline"
832         ]["test accuracies"],
833         campaign_results["conditions"]["vector_rag"]["
test accuracies"],
834         campaign_results["conditions"]["rar_compressed"]["
test accuracies"]
835     ]
836     ax1.boxplot(data_to_plot, labels=['Baseline', 'Vector
RAG', 'RAR (Compressed)'])
837     ax1.set_title(r"$\mathbf{Test\ Vault\ Accuracies\ (N
=10\ Seeds)}$", fontweight='bold')
838     ax1.set_ylabel("Strict Test Accuracy", fontweight='bold
')
839
840     # Right Plot: Context densities and Net Tokens Dual-
Axis
841     ax2.grid(True, linestyle='--', alpha=0.6)
842     x = np.arange(3)
843     densities_mean = [
844         np.mean(campaign_results["conditions"]["
stateless_baseline"]["prompt densities"]),
845         np.mean(campaign_results["conditions"]["vector_rag"
846         ]["prompt densities"]),
847         np.mean(campaign_results["conditions"]["
rar_compressed"]["prompt densities"])
848     ]
849     tokens_mean = [
850         np.mean(campaign_results["conditions"]["
stateless_baseline"]["net tokens"]),
851         np.mean(campaign_results["conditions"]["vector_rag"
852         ]["net tokens"]),
853         np.mean(campaign_results["conditions"]["
rar_compressed"]["net tokens"])
854     ]
855     ax2.bar(x - 0.2, densities_mean, width=0.35, label="
Context Density", color='#1c4f8a')
856     ax2.set_ylabel(r"Mean Context Density ($\delta$)",
fontweight='bold')
857     ax2.set_xticks(x)
858     ax2.set_xticklabels(['Baseline', 'Vector RAG', 'RAR'])
859     ax2.set_title(r"$\mathbf{Context\ and\ Token\ Tradeoffs
}$", fontweight='bold')
860
861     ax2_twin = ax2.twinx()
862     ax2_twin.bar(x + 0.2, tokens_mean, width=0.35, label="
Net Tokens", color='#b41e1e')
863     ax2_twin.set_ylabel("Net Tokens (chars)", fontweight='
bold')
864
865     ax2.legend(loc='upper left', bbox_to_anchor=(0.0,
-0.15))
866     ax2_twin.legend(loc='upper right', bbox_to_anchor=(1.0,
-0.15))
867
868     plt.tight_layout()
869     plt.savefig(os.path.join(OUTPUT_DIR, "
fig2_crs_trajectory.png"), dpi=300)
870     plt.close()
871
872     # Re-plot density comparisons alone
873     plt.figure(figsize=(7, 4.5))

```

```

870 plt.grid(True, linestyle='--', alpha=0.6)
871 plt.bar(['Baseline', 'Vector RAG', 'RAR'],
densities_mean, color=['#b41e1e', '#1c4f8a', '#2ca02c'
])
872 plt.ylabel(r"Mean Context Density ( $\delta$ )",
fontweight='bold')
873 plt.title(r" $\mathbf{Empirical\ Context\ Density\ comparison}$ ", fontweight='bold')
874 plt.savefig(os.path.join(OUTPUT_DIR, "fig3_density.png"
), dpi=300)
875 plt.close()
876
877 print("Scientific visual assets saved directly to the
workspace.")
878
879 if __name__ == "__main__":
880     asyncio.run(execute_campaign())

```

D. Sample Execution Logs

The empirical results across the $N = 10$ randomized physical seeds are fully reproducible from the serialized local `pilot_results.json` SRE state.

```

1 {
2   "meta": "Upgrade campaign with PyTorch Residual MLP
hyperparameter search on dynamic synthetic manifold",
3   "SEEDS": [
4     42,
5     7,
6     13,
7     23,
8     88,
9     99,
10    101,
11    107,
12    113,
13    127
14  ],
15   "CYCLES": 60,
16   "wilcoxon_p_value_RAR_vs_Baseline": "0.2461",
17   "data": {
18     "dataset": "Dynamic Synthetic Manifold (PyTorch
Residual MLP)"
19   }
20 }

```

Listing 2. Snippet of `pilot_results.json` Logging